Brickstorm: Software and Symbolic AI — Content V2.0

Foundations of CS & AI 2026 · Team Brickstorm · Snapshot 3 November 2026

Rendered from the Markdown source in `content/`. Contents: module outline; Chapter 3 (topics 01–06); Chapter 4 (topics 07–13).

Module Outline: Software and Symbolic AI

Version: V2.0 snapshot, 3 November 2026 (V1.0: 24 October 2026)
Module: Chapter 3 (Software & Algorithms) and Chapter 4 (Symbolic AI), Foundations of CS & AI 2026

Intended Learning Outcomes

After the whole module, a beginner can:

1. Explain system vs application software and what an operating system does.
2. Describe how programming languages, software engineering and databases support software development.
3. Carry out basic searching, sorting and recursive algorithms by hand and explain basic data structures.
4. Explain what Symbolic AI is and how it represents knowledge with logic, facts and rules.
5. Show step by step how a rule-based system and a search procedure reach a conclusion.
6. Discuss real-world uses, strengths and limits of Symbolic AI.

Prerequisite Knowledge

Basic computer use (files, browsers, apps). No programming, logic or mathematics beyond secondary school. Each topic lists its own prerequisites, and every new term is defined where it first appears.

Learning Sequence and Development Status

Status labels: **Outline only** (objectives and plan, no teaching text yet) · **Drafted** (full text with example, activity and answer, not yet reviewed) · **Checked** (reviewed by a second member) · **Needs revision**. A topic becomes "Checked" only when the review is recorded in the Stage 2 logs.

#	Topic	Main worked example and activity	Learning outcome	Status
01	Software Overview and OS	Double-click walk-through; round-robin scheduling trace; sorting programs	1	Drafted V2.0
02	Programming Languages	Tracing a five-line program; predict the output; find the error	2	Drafted V2.0
03	Software Engineering	Study-timer app through the life cycle; which stage should have caught it	2	Drafted V2.0
04	Databases and SQL	Queries on a Students table; predict the rows	2	Drafted V2.0
05	Algorithms	Binary search, bubble sort, recursion traced by hand	3	Drafted V2.0

#	Topic	Main worked example and activity	Learning outcome	Status
06	Data Structures	Back button as a stack; BST search; choosing structures	3	Drafted V2.0
07	Intro to Symbolic AI	Animal-guessing rules; classify AI systems	4, 6	Drafted V2.0
08	Logic and Reasoning	Truth tables; modus ponens and tollens; invalid patterns	4	Drafted V2.0
09	Knowledge Representation	Semantic network with inheritance; frames	4	Drafted V2.0
10	Rule-Based and Expert Systems	Laptop diagnosis with forward and backward chaining	5	Drafted V2.0 (pilot)
11	Planning and Search	BFS vs DFS on a small map; preconditions and effects	5	Drafted V2.0
12	Symbolic NLP	Parsing "The cat sat"; keyword chatbot failures	4, 6	Drafted V2.0
13	Hands-on and Review	Six-rule study-method advisor; module review quiz	5, 6	Drafted V2.0

How the topics connect. Topics 01–06 build from "what is software" (01) through "how programs are written, built and store data" (02–04) to "how programs search, sort and organise data" (05–06). Topics 07–13 build from "what is Symbolic AI" (07) to logic (08), how knowledge is stored (09), how rules reason (10) and how search finds plans (11), then apply rules to language (12) and bring everything together (13). Key links: lists and binary search (05) need sorted arrays (06); the call stack in recursion (05) is a stack (06); queues and stacks (06) are the frontiers of BFS and DFS (11); trees (06) reappear as decision trees (07) and parse trees (12); logic programming (02) and declarative SQL (04) preview rules (08, 10). Topic 10 is the centre of Chapter 4 because it combines facts, rules and search.

Every topic file uses the same template: Learning Objectives · Explanation (with key terms) · Worked Example · Activity · Answer · Common Misunderstandings · Quick Self-Check · How This Topic Will Be Reused · References.

Gaps and Uncertainties

- All 13 topics are drafted, but **none is "Checked" yet**: second-member reviews and beginner tests must be recorded in the Stage 2 logs (tasks T16, T22).
- Exact depth expected for Chapters 3 and 4 still needs confirming with the teacher (T9). Topics 03, 12 and 13 are kept brief as planned.
- Page or section numbers for textbook references still need confirming by the team.
- Optional extensions not covered (clearly outside the core scope): Big O notation beyond an intuitive mention, SQL JOIN syntax, merge sort and quicksort, heuristic search (A*), probabilistic reasoning.

01 Software Overview and OS

Chapter: Chapter 3: Software & Algorithms
Status: Drafted (V2.0, 3 Nov 2026). Examples checked by running a short program. Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: None (basic computer use: files, apps, a browser).
Draft note: Prepared with Claude assistance (AI-use record AI-5). The round-robin trace was run with a short program. Team members must review and be able to explain every part.

Learning Objectives

After this topic, a beginner can:

1. Distinguish **system software** from **application software** and give examples of each.
2. List the main jobs of an **operating system (OS)**: managing processes, memory, files and devices, and providing a user interface.
3. Explain in simple steps what happens when you open a document.
4. Trace how an OS shares one processor between several programs using **round-robin scheduling**.
5. Describe in a few sentences how operating systems developed (batch → time-sharing → personal → mobile).

Explanation

Hardware and software

A computer has two sides:

- **Hardware** is the physical part you can touch: the processor (CPU), memory (RAM), storage (SSD or hard disk), screen, keyboard.
- **Software** is the set of instructions, called **programs**, that tells the hardware what to do.

Hardware without software can do nothing useful. Software without hardware has nothing to run on.

Two kinds of software

	System software	Application software
Purpose	Runs and looks after the computer itself	Helps the user do a task
Who uses it directly	Mostly other programs; the user rarely notices it	The user
Examples	Operating systems (Windows, macOS, Linux, Android, iOS); **device drivers** (small programs that let the OS talk to a printer or graphics card); **utilities** (antivirus, disk cleanup, backup tools)	Word processors, web browsers, Weixin (WeChat), games, music players, Photoshop

	System software	Application software
Can the computer work without it?	No (without an OS, apps cannot run)	Yes (the computer still runs; you just cannot do that task)

A simple test: "*Does this program do a job for me, or does it keep the computer working?*" The first is application software; the second is system software.

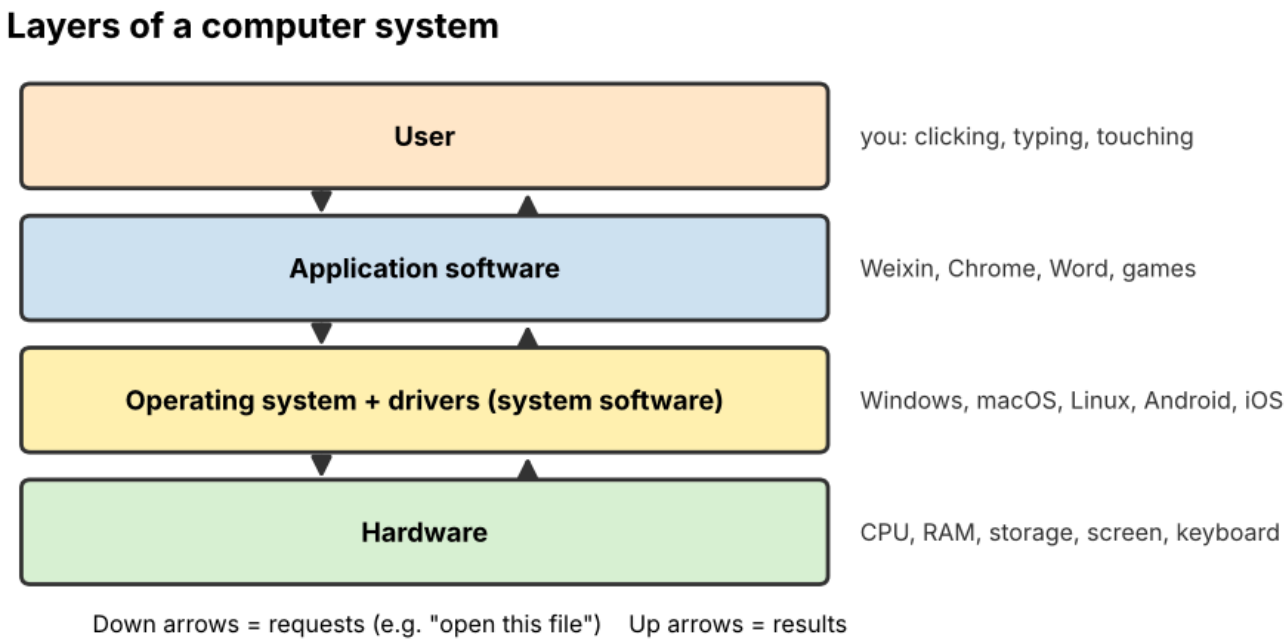


Figure 1. The layers of a computer system. The user works with applications. Applications ask the operating system for what they need (memory, files, the screen). Only the OS and drivers talk to the hardware directly.

What an operating system does

The **operating system** is the main piece of system software. It sits between the applications and the hardware, like a manager in a busy office who shares out rooms, desks and the printer so that workers do not fight over them.

OS job	What it means	Everyday example
Process management	Starts and stops programs and shares the CPU between them	Music keeps playing while you type a document
Memory management	Gives each running program its own part of RAM and takes it back when the program closes	Closing a big game frees memory, so other apps run faster
File management	Organises data into files and folders on storage and controls who may open them	Saving `essay.docx` in `Documents`
Device management	Controls input and output devices through drivers	Plugging in a USB mouse and it simply works

OS job	What it means	Everyday example
User interface	Gives the user a way to control the computer: a graphical interface (windows, icons) or a command line	The desktop, the Start menu, the phone home screen
Security	Checks who is using the computer and what they are allowed to do	Login password, app permissions on a phone

Processes and the CPU

A **program** is a file of instructions sitting on storage. When you start it, it becomes a **process**: a program that is running, with its own memory and its own place in the CPU's queue.

A simple processor core can carry out only one process at a time, yet you see many apps working "at once". The OS creates this illusion by switching very quickly between processes, many times per second. Each process moves between three main **states**:

- **Ready:** waiting for its turn on the CPU.
- **Running:** using the CPU now.
- **Waiting (blocked):** paused until something happens, such as a file finishing loading or the user pressing a key.

One simple and fair way to share the CPU is **round-robin scheduling**. Each process gets a short, fixed slice of time called a **time quantum**. When its slice ends, it goes to the back of the ready queue (if it still has work), and the next process runs.

Memory

RAM is fast working memory, but it is limited and is emptied when the power goes off. Storage (SSD) is slower but keeps data permanently. When you open a program, the OS copies the parts it needs from storage into RAM. The OS also keeps processes apart, so one crashing app cannot overwrite another app's memory. When RAM is full, many operating systems use **virtual memory**: they move parts of RAM that are not being used right now out to storage, then bring them back when needed. This lets more programs run, but too much of it makes the computer slow.

How operating systems developed

Period	Kind of OS	Main idea
1950s	**Batch** systems	Jobs on punched cards were collected and run one after another; no interaction while a job ran
1960s–1970s	**Time-sharing** systems, such as **UNIX** (Bell Labs, 1969)	Many users at terminals share one computer; the OS switches between them quickly
1980s–1990s	**Personal computer** OS with graphical interfaces (Mac OS 1984, Windows)	One user, windows and a mouse
1991 onward	**Linux**, a free UNIX-like system	Open source; now runs most web servers and is the base of Android
2007 onward	**Mobile** OS (iOS 2007, Android 2008)	Touch screens, app stores, strong control of battery and app permissions

UNIX is worth remembering: its ideas (files in a tree of folders, small tools combined together, a command line called a *shell*) live on in Linux, macOS and Android.

Worked Example 1: What happens when you double-click

`essay.docx`

1. **User interface.** The OS notices the double-click on the file icon.
2. **File management.** The OS finds `essay.docx` in the folder tree and sees from the `.docx` ending that Word (or another word processor) opens it.
3. **Process management.** The OS starts the word processor as a new process and puts it in the ready queue.
4. **Memory management.** The OS gives the new process a block of RAM and copies the program's instructions and the document's contents into it.
5. **Device management.** The OS, through the graphics driver, draws the program's window on the screen and sends your key presses to it.
6. **Security.** If the file belongs to another user, the OS checks whether you are allowed to open it before step 2 continues.

Notice that the word processor never touches the disk or the screen directly. It asks the OS, and the OS does the work.

Worked Example 2: Round-robin scheduling

Three processes are ready, in this order. Each number is how many milliseconds (ms) of CPU work it still needs. The time quantum is **2 ms**.

Process	CPU time needed
Music	3 ms
Browser	5 ms
Editor	2 ms

Time (ms)	Running	What happens next	Ready queue afterwards
0–2	Music	Used 2 ms, needs 1 more → back of queue	Browser, Editor, Music
2–4	Browser	Used 2, needs 3 more → back of queue	Editor, Music, Browser
4–6	Editor	Used 2, needs 0 → **finishes at 6**	Music, Browser
6–7	Music	Used its last 1 ms → **finishes at 7**	Browser
7–9	Browser	Used 2, needs 1 more → back of queue	Browser
9–10	Browser	Used its last 1 ms → **finishes at 10**	(empty)

Finishing order: **Editor (6 ms), Music (7 ms), Browser (10 ms)**. The short job (Editor) finished early even though it was last in the queue. This is why round-robin feels responsive: no single long job can hold the CPU for long.

Activity

1. **Sort the programs.** Label each as **S** (system software) or **A** (application software): (a) Windows 11, (b) Weixin, (c) a printer driver, (d) Google Chrome, (e) antivirus scanner, (f) Excel, (g) Android, (h) a disk-cleanup tool, (i) Spotify, (j) a calculator app.
2. **Match the OS job.** Which OS job (process, memory, file, device, user interface, security) is shown in each situation?
    - (a) A phone asks "Allow this app to use your camera?"
    - (b) You plug in headphones and the sound moves to them.
    - (c) You drag a photo into a new folder.
    - (d) A video keeps downloading while you scroll a web page.
    - (e) A computer with many tabs open slows down, and closing tabs speeds it up.
3. **Trace round-robin.** Processes in the queue: **Game 4 ms, Chat 1 ms, Upload 3 ms**. Time quantum = 2 ms. Fill in a table like Worked Example 2. In what order do they finish, and at what times?
4. **Explain a decision.** A friend says: "The OS is just the desktop picture and icons." Write two sentences explaining why this is incomplete.

Answer

1. S: (a) Windows 11, (c) printer driver, (e) antivirus, (g) Android, (h) disk cleanup. A: (b) Weixin, (d) Chrome, (f) Excel, (i) Spotify, (j) calculator. *Note:* antivirus and disk cleanup are **utilities**, which most textbooks class as system software because they look after the computer rather than do a user task.
2. (a) Security. (b) Device management. (c) File management. (d) Process management (two processes share the CPU). (e) Memory management (each tab uses RAM; when RAM is full, the OS moves data to slower storage).
3. 0–2 Game (needs 2 more, to back) → queue Chat, Upload, Game; 2–3 Chat (**finishes at 3**) → queue Upload, Game; 3–5 Upload (needs 1 more) → queue Game, Upload; 5–7 Game (**finishes at 7**) → queue Upload; 7–8 Upload (**finishes at 8**). Order: **Chat 3 ms, Game 7 ms, Upload 8 ms.**
4. The desktop is only the **user interface**, one of the OS's jobs. The OS also runs and switches between programs, shares out memory, stores files and controls devices, mostly without the user seeing it.

Common Misunderstandings

- "*An app and a process are the same thing.*" An app is the program file; a process is a running copy of it. Opening two Chrome windows can create several processes from one program.
- "*More RAM makes the CPU faster.*" More RAM lets more programs stay in fast memory, which avoids slowdowns, but it does not make the processor itself calculate faster.
- "*Linux and UNIX are the same.*" Linux was written from scratch in 1991 to *behave like* UNIX; it does not contain the original UNIX code.

Quick Self-Check

1. Is a device driver system or application software? *(System software.)*
2. Name the OS job that decides which program uses the CPU next. *(Process management / scheduling.)*

3. In round-robin, what happens to a process whose time slice ends before it has finished? *(It goes to the back of the ready queue.)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** chapter "Software and the Operating System"; questions on sorting programs, matching OS jobs, a round-robin trace.
- **PPT Slides:** Figure 1 (layers), the OS-jobs table, the double-click steps, the round-robin table built row by row.
- **Video Tutorial:** "What an operating system does" (planned tutorial 1): narrate the double-click walk-through, then the round-robin trace.

References

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley. (OS functions, process states, round-robin scheduling, virtual memory.)
2. Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Pearson. (History of operating systems.)
3. Ritchie, D. M., & Thompson, K. (1974). The UNIX time-sharing system. *Communications of the ACM, 17*(7), 365–375. (UNIX.)
4. Foundations of CS & AI 2026, Chapter 3 course slides ("Software overview": system and application software; operating systems: evolution, process management, memory management).

The double-click walk-through and the scheduling examples are the team's own.

02 Programming Languages

Chapter: Chapter 3: Software & Algorithms
Status: Drafted (V2.0, 3 Nov 2026). All programs were run and their outputs checked. Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: Topic 01 (what a program and a CPU are).
Draft note: Prepared with Claude assistance (AI-use record AI-5). Every program in this file was run in Python 3 and the outputs below are the real outputs.

Learning Objectives

After this topic, a beginner can:

1. Explain why programming languages exist and name the three levels: **machine code**, **assembly language** and **high-level languages**.
2. Describe the difference between a **compiler** and an **interpreter**.
3. Name four **categories** (paradigms) of languages and give one example of each.
4. Read and trace a short program that uses a **variable**, a **condition** and a **loop**, and predict its output.

Explanation

Why we need programming languages

A CPU only understands **machine code**: long patterns of 0s and 1s, each pattern meaning one tiny instruction such as "add these two numbers". People find this almost impossible to read or write. A **programming language** is a precise, limited language that people can write and that can be translated into machine code. Unlike English, every statement has exactly one meaning, so the computer never has to guess.

Three levels of language

Level	What it looks like	Pros	Cons
Machine code (1940s)	`10110000` `01100001`	Runs directly on the CPU; no translation	Unreadable; different for each type of CPU
Assembly language (late 1940s–1950s)	`LOAD A · ADD B · STORE C`	Short words (*mnemonics*) instead of binary; still very fast	One line ≈ one machine instruction, so programs are long; still tied to one CPU type
High-level language (1957 onward)	`c = a + b`	Close to human language and maths; one line does a lot; runs on many kinds of computer	Must be translated; the programmer has less direct control of the hardware

(The machine-code and assembly lines above are simplified illustrations, not the code of a real CPU.)

The first widely used high-level language was **FORTRAN** (IBM, 1957), for science and engineering. **COBOL** (1959) was designed for business records. Later came **C** (1972), **C++** (1985), **Python** (1991), **Java** (1995) and **JavaScript** (1995).

Translating: compiler vs interpreter

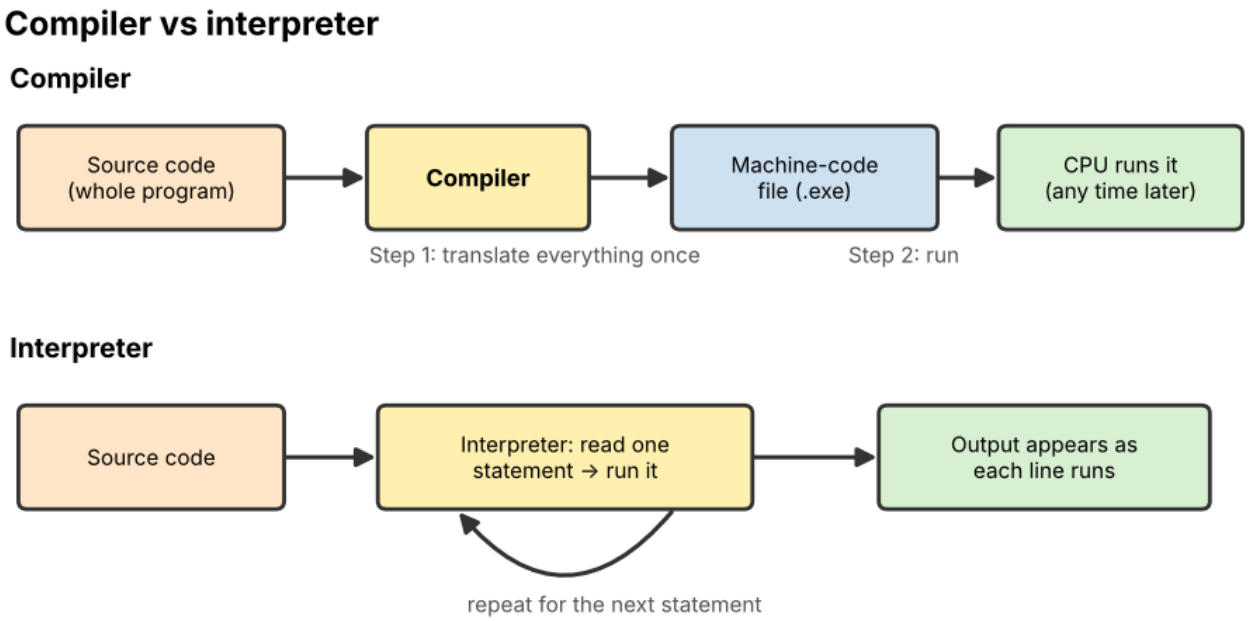


Figure 1. A compiler translates the whole program once, before it runs. An interpreter translates and runs it one statement at a time.

	Compiler	Interpreter
How it works	Translates the whole **source code** into a machine-code file first; then that file is run	Reads one statement, runs it, then reads the next
Errors	Reports many errors before anything runs	Stops at the first error it reaches while running
Speed of the finished program	Usually faster	Usually slower
Typical examples	C, C++, Go, Rust	Python, JavaScript (in practice)
Analogy	Translating a whole book before printing it	A live interpreter at a meeting, sentence by sentence

In reality many modern languages mix the two (Java and Python first turn code into an in-between form called *bytecode*). For beginners, the key idea is **translate everything first** vs **translate as you go**.

Categories of languages (paradigms)

A **paradigm** is a style of writing programs.

Paradigm	Main idea	Example languages	Tiny example idea
Procedural / imperative	A list of commands carried out in order, grouped into procedures	C, Pascal, (Python can do this)	"Do step 1, then step 2, then repeat step 3"
Object-oriented	Programs are built from **objects** that combine data and actions	Java, C++, Python	A `Student` object has a name and a method `enrol()`
Functional	Programs are built from functions that return results without changing stored data	Haskell, Lisp	`total = sum(map(square, numbers))`
Logic / declarative	You state facts and rules; the system works out the answer	**Prolog**, SQL	`parent(tom, ann).` then ask `?- parent(tom, X).`

Logic programming connects directly to Chapter 4: **Prolog** (1972) was built for Symbolic AI, and its facts and rules work like the rule-based systems in Topic 10. **SQL** (Topic 04) is also declarative: you say *what* data you want, not *how* to find it.

Common features of high-level languages

Almost every language you will meet has these building blocks. Examples use Python.

Feature	Meaning	Python example
Variable	A named box that stores a value; the value can change	`score = 0`
Data type	The kind of value: whole number (integer), decimal number, text (string), True/False (Boolean)	`5`, `3.2`, `"Anu"`, `True`
Assignment	Put a value into a variable. = means "store", not "is equal"	`score = score + 1`
Condition (selection)	Do something only if a test is true	`if score > 50:`
Loop (iteration)	Repeat some statements	`for n in [4, 7, 2]:` or `while x > 1:`
Function	A named group of statements that can be reused	`def area(w, h): return w * h`
Comment	A note for humans, ignored by the computer	`# this is a comment`

In Python, the lines that belong to an `if` or a loop are **indented** (moved right). Indentation is part of the language, not decoration.

Worked Example: Tracing a five-line program

```
total = 0
for n in [4, 7, 2]:
    if n > 3:
        total = total + n
print(total)
```

What it does in words: start a running total at 0; look at each number in the list; if the number is bigger than 3, add it to the total; finally show the total.

To **trace** a program means to follow it line by line and write down the value of every variable. This is how programmers check what a program will do before (or instead of) running it.

Step	Line running	n	Test `n > 3`	total
1	`total = 0`	–	–	0
2	loop: first item	4	4 > 3 is **True**	0 + 4 = **4**
3	loop: second item	7	7 > 3 is **True**	4 + 7 = **11**
4	loop: third item	2	2 > 3 is **False**	stays **11**
5	`print(total)`	–	–	prints **11**

Output: 11

Activity

1. **Predict the output** of Program A. Then change `== 0` to `== 1` and predict again.

```
count = 0
for i in range(1, 6):      # i takes the values 1, 2, 3, 4, 5
    if i % 2 == 0:         # % gives the remainder after dividing
        count = count + 1
print(count)
```

2. **Predict the output** of Program B. Write the value of `x` after each pass of the loop. (`//` divides and rounds down, so `7 // 2` is 3.)

```
x = 10
while x > 1:
    x = x // 2
    print(x)
```

3. **Find the error.** A student wants to add up only the numbers **below** 5 in `[4, 7, 2]` and expects the answer 6. Their program prints 7. What did they get wrong?

```
total = 0
for n in [4, 7, 2]:
    if n > 5:
        total = total + n
print(total)
```

4. **Compare.** You are writing (a) a game that must run as fast as possible, and (b) a quick script to rename 50 photos once. Would a compiled or an interpreted language suit each better? Give one reason for each.

5. **Classify.** Which paradigm best fits each line: (a) `SELECT name FROM Students WHERE year = 1` (b) a `Car` object with `speed` and `brake()` (c) `likes(anu, tea).`

Answer

1. Program A counts the even numbers from 1 to 5: i = 2 and i = 4 → output **2**. With `== 1` it counts odd numbers 1, 3, 5 → output **3**.
2. Pass 1: 10 // 2 = **5** (5 > 1, continue). Pass 2: 5 // 2 = **2** (2 > 1, continue). Pass 3: 2 // 2 = **1** (1 > 1 is false, stop). Output, one per line: **5, 2, 1**.
3. The condition is `n > 5`, so only 7 is added and the result is 7. It should be `n < 5`, which adds 4 + 2 = **6**. This kind of mistake is called a **logic error**: the program runs, but does the wrong thing.
4. (a) **Compiled**: the whole program is translated to machine code in advance, so it usually runs faster. (b) **Interpreted**: you can write and run it immediately without a build step; speed does not matter for a one-time job.
5. (a) Declarative (SQL). (b) Object-oriented. (c) Logic (Prolog fact).

Common Misunderstandings

- "`=` *means equals.*" In most languages `=` means **store**. `total = total + n` would be nonsense in maths but is normal in programming: work out the right side, then store it on the left. Testing equality uses `==`.
- "*High-level means advanced or difficult.*" It means *far from the hardware* and closer to human language, which usually makes it easier.
- "*A program that runs without crashing is correct.*" Activity 3 runs perfectly and still gives the wrong answer.

Quick Self-Check

1. Which translator reports errors in the whole program before it runs? *(A compiler.)*
2. Which paradigm does Prolog belong to, and which later topic uses the same idea? *(Logic programming; Topic 10, rule-based systems.)*
3. What is printed by `x = 3` then `x = x * 2` then `print(x)`? *(6)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** chapter "Programming Languages"; predict-the-output and find-the-error questions from the activity.
- **PPT Slides:** three-levels table, Figure 1, paradigm table, the trace table revealed row by row.
- **Video Tutorial:** not selected as a separate video; the trace table may be reused as a short clip inside the algorithms tutorial.

References

1. Brookshear, J. G., & Brylow, D. (2019). *Computer Science: An Overview* (13th ed.). Pearson. (Language generations, translation, paradigms.)
2. Sebesta, R. W. (2016). *Concepts of Programming Languages* (11th ed.). Pearson. (History of FORTRAN, COBOL and later languages; compilers and interpreters.)
3. Python Software Foundation. (2024). *The Python Tutorial*. https://docs.python.org/3/tutorial/ (Variables, `if`, `for`, `while`, `range`.)

4. Clocksin, W. F., & Mellish, C. S. (2003). *Programming in Prolog* (5th ed.). Springer. (Logic programming.)
5. Foundations of CS & AI 2026, Chapter 3 course slides ("Programming languages: evolution, categories, features").

All programs and traces are the team's own examples.

03 Software Engineering

Chapter: Chapter 3: Software & Algorithms
Status: Drafted (V2.0, 3 Nov 2026). Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: Topic 02 (what a program is; that programs can contain errors).
Depth: Brief introduction (B) as agreed in the project plan: the main ideas, one worked example and one activity, without detailed methodologies.
Draft note: Prepared with Claude assistance (AI-use record AI-5). Team members must review and be able to explain every part.

Learning Objectives

After this topic, a beginner can:

1. Name and order the stages of the **software development life cycle (SDLC)**.
2. Compare the **waterfall** and **agile** process models and say when each fits.
3. Explain what **modularity** is and why it helps.
4. Explain why **testing** and **maintenance** matter, and write simple test cases.
5. Describe what **version control** (such as Git and GitHub) does and why teams use it.

Explanation

Why "engineering"?

Writing a 10-line program for yourself is easy. Building an app used by millions, written by hundreds of people over years, is not. **Software engineering** is the organised, disciplined way of building software so that it does what users need, works reliably, and can be changed later. Just as civil engineers plan, design and inspect a bridge before and after building it, software engineers plan, design, test and maintain software.

The software development life cycle (SDLC)

Stage	Key question	Output
1. **Requirements**	*What* must the software do, and for whom?	A list of requirements, agreed with users
2. **Design**	*How* will it be organised? Which parts (modules), screens and data?	Diagrams, screen sketches, module plan
3. **Implementation (coding)**	Write the program	Source code
4. **Testing**	Does it work correctly? Does it meet the requirements?	Test results, bug reports, fixes
5. **Deployment**	Put it in users' hands	Released app or website
6. **Maintenance**	Fix bugs found later, adapt to new devices, add features	Updates (version 1.1, 1.2, 2.0 …)

A **requirement** can be *functional* (what the system does: "the timer counts down 25 minutes") or *non-functional* (how well it does it: "the app opens in under 2 seconds").

Errors are much cheaper to fix early. A misunderstanding found while writing requirements costs a conversation; the same misunderstanding found after release can mean redesigning, recoding, retesting and re-releasing.

Two process models

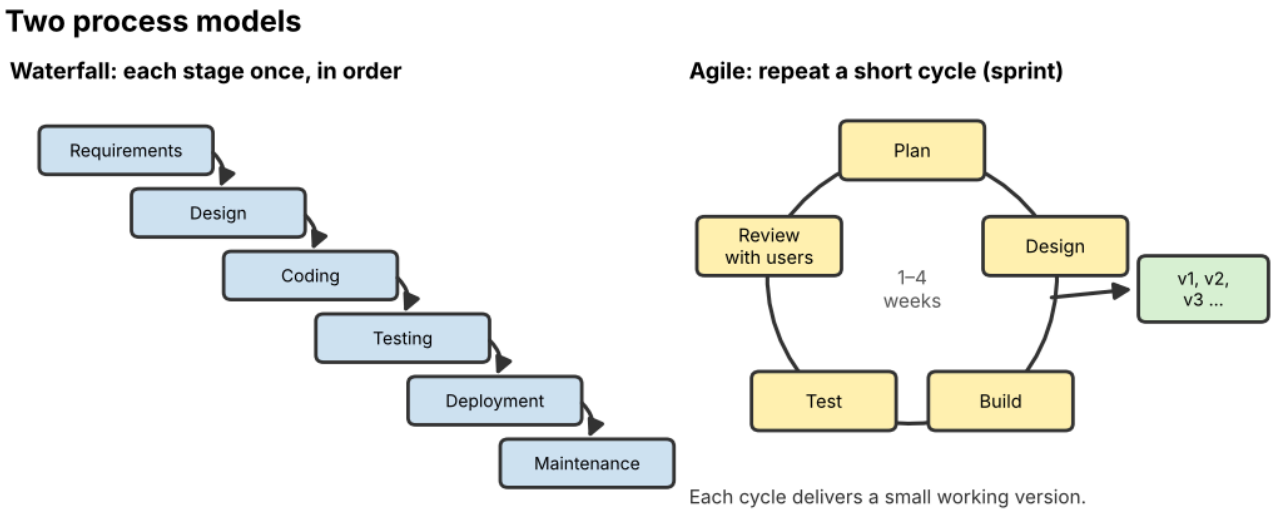


Figure 1. Waterfall goes through each stage once, in order. Agile repeats a short cycle many times, delivering a small working piece of software each time.

	Waterfall	Agile (for example, Scrum)
How it flows	Each stage is finished and signed off before the next starts	Work is split into short cycles (**iterations** or **sprints**, often 1–4 weeks); each cycle plans, builds, tests and shows a working piece
Requirements	Fixed at the start	Expected to change; updated after each cycle
User involvement	Mainly at the start and end	Frequent: users see and comment on every cycle
Fits well when	Requirements are clear and stable; changes are costly (e.g. safety systems)	Requirements are unclear or change often (e.g. a new phone app)
Main risk	Problems in requirements are found very late	Less predictable final scope and date; needs disciplined teams

The *Manifesto for Agile Software Development* (2001) summed up the agile values, such as "responding to change over following a plan".

Modularity

Modularity means splitting a program into separate parts, called **modules**, each doing one job and connecting to the others through a clear **interface** (the agreed way of calling it).

Think of LEGO bricks: each brick is simple, and the studs are the interface that lets bricks fit together.

Benefits: different people can build different modules at the same time; each module can be tested on its own; a module can be replaced or reused without rewriting everything.

Quality assurance and testing

Quality assurance (QA) is everything a team does to make sure the software is good enough: reviews, standards and, above all, **testing**.

- A **bug** is an error in the software.
- A **test case** gives an **input**, the **expected output**, and then compares it with the **actual output**. If they differ, the test fails and a bug has been found.
- Levels of testing: **unit testing** (one module alone) → **integration testing** (modules working together) → **system testing** (the whole program) → **acceptance testing** (real users check that it meets their requirements).
- Good testers also try **boundary** and **unexpected** inputs: zero, negative numbers, empty fields, very large values.

Version control

Version control records every change to a project's files, who made it, when and why. **Git** is the most widely used version-control tool; **GitHub** is a website that stores Git projects online so a team can share them.

Term	Meaning	In our team
Repository (repo)	The project folder plus its full history	`BRICKSTORM-AI`
Commit	A saved snapshot of changes with a message explaining them	"Add worked example to Topic 04"
Branch	A separate line of work, so you can change things without disturbing the main version	A branch for drafting Topic 11
Merge	Combine a branch's changes back into the main branch	Merging the Topic 11 branch into `main`
Pull request	A request on GitHub to merge a branch, where teammates review the changes first	Our rule: another member reviews before merging

Why teams use it: nobody overwrites anyone else's work; mistakes can be undone by going back to an earlier commit; the history shows who contributed what. This module's own repository is a real example: each topic was drafted and revised through commits.

Worked Example: A study-timer app through the life cycle

Stage	What happens for the study timer
Requirements	Functional: the user can start a 25-minute study session and a 5-minute break; an alarm sounds at the end; the app counts sessions finished today. Non-functional: works on a phone; easy to use with one hand.

Stage	What happens for the study timer
Design	Modules: **Timer** (counts down), **Alarm** (plays the sound), **History** (stores the number of sessions), **Screen** (buttons and display). Sketch of one screen with a big Start button.
Implementation	Each module is coded; the Screen calls Timer's `start(25)`.
Testing	Unit test of Timer: input `start(1)` → expected: reaches 0 after 60 seconds. Boundary test: what if the user presses Start twice? Expected: the timer does not run twice as fast. Acceptance test: five classmates use it for a day.
Deployment	Released to an app store as version 1.0.
Maintenance	Users report the alarm is silent when the phone is on vibrate (bug fix in 1.1); users ask for custom session lengths (new feature in 1.2).

Waterfall or agile? For a small app whose users will have new ideas once they try it, agile fits better: release a basic timer in the first sprint, then add history and custom lengths in later sprints based on feedback.

Activity

1. **Which stage should have caught it?** For each problem, name the SDLC stage where it should have been prevented or found, and say why.
    - (a) A finished library app lets students borrow books, but the library actually needed it to handle *room bookings*.
    - (b) The app crashes when a user leaves the "age" box empty.
    - (c) An app built for Android 12 stops working after a phone update to a newer version.
    - (d) Two programmers wrote modules that both work alone, but together the dates come out in different formats (day/month vs month/day).
    - (e) Every screen has a different layout and the app is confusing to use.
2. **Write test cases.** A function `grade(score)` should return "Pass" for scores 60–100 and "Fail" for 0–59. Write four test cases (input, expected output), including at least two boundary values.
3. **Choose a model.** For each project, choose waterfall or agile and give one reason: (a) control software for a hospital radiation machine, where every requirement is set by law; (b) a new social app for students, where the team is not sure which features people will like.
4. **Version control.** Two teammates both edit Topic 05 on the same day. Explain how branches and a pull request stop one person's work from overwriting the other's.

Answer

1. (a) **Requirements**: the team built the wrong thing because it never confirmed what the users needed. (b) **Testing** (unit/system testing with an empty or unexpected input). (c) **Maintenance**: adapting software to a changed environment is normal maintenance work. (d) **Integration testing** (and a clearer interface in design: agree one date format). (e) **Design**: a consistent screen design should have been planned; acceptance testing with users would also reveal it.

2. Example answer: `grade(75)` → "Pass"; `grade(30)` → "Fail"; `grade(60)` → "Pass" (boundary); `grade(59)` → "Fail" (boundary). Good extra cases: `grade(0)` → "Fail", `grade(100)` → "Pass", and an invalid input such as `grade(-5)` or `grade(120)` → the requirements must say what should happen (for example, an error message). Spotting that the requirement is missing is a good answer.
3. (a) **Waterfall**: the requirements are fixed and well known, and every change must be carefully checked and documented for safety. (b) **Agile**: the requirements are uncertain, so short cycles with user feedback let the team change direction cheaply.
4. Each person works on their **own branch**, so their commits do not change `main`. When finished, each opens a **pull request**; a teammate reviews the changes, and Git merges them. If both changed the *same lines*, Git reports a **merge conflict** and a person decides which version to keep, instead of one silently overwriting the other.

Common Misunderstandings

- "*Testing proves the program has no bugs.*" Testing can show that bugs exist, but it cannot prove there are none; it only checks the cases you tried.
- "*Maintenance is just fixing bugs.*" It also includes adapting to new systems and adding features, and over the life of a program it is often the largest share of the work.
- "*Agile means no planning.*" Agile teams plan constantly, just in short cycles instead of one big plan at the start.
- "*Git and GitHub are the same.*" Git is the version-control tool on your computer; GitHub is one website that hosts Git repositories.

Quick Self-Check

1. Put in order: testing, requirements, maintenance, design, coding. *(Requirements, design, coding, testing, maintenance.)*
2. Which model delivers working software in short repeated cycles? *(Agile.)*
3. What does a commit message record? *(What changed and why.)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** short chapter; questions from the "which stage?" activity and the test-case task.
- **PPT Slides:** SDLC table, Figure 1, study-timer table.
- **Video Tutorial:** not selected for video (brief topic).

References

1. Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson. (Life cycle, waterfall and agile models, testing levels, maintenance.)
2. Beck, K., et al. (2001). *Manifesto for Agile Software Development*. https://agilemanifesto.org/
3. Chacon, S., & Straub, B. (2014). *Pro Git* (2nd ed.). Apress. https://git-scm.com/book (Commits, branches, merging.)
4. Foundations of CS & AI 2026, Chapter 3 course slides ("Software engineering: development process models, modularity, quality assurance").

The study-timer example, the activity problems and the `grade` function are the team's own.

04 Databases and SQL

Chapter: Chapter 3: Software & Algorithms
Status: Drafted (V2.0, 3 Nov 2026). Every query was run in SQLite and the results below are the real results. Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: Topic 01; the idea of a table with rows and columns (like a spreadsheet).
Draft note: Prepared with Claude assistance (AI-use record AI-5). All tables and queries were loaded into SQLite and run to check the answers.

Learning Objectives

After this topic, a beginner can:

1. Explain what a **database**, a **table**, a **row** (record) and a **column** (field) are.
2. Explain what a **primary key** and a **foreign key** are, and how they link tables.
3. Name the three classic **database models** (hierarchical, network, relational).
4. Read and write basic **SQL** queries with `SELECT`, `FROM`, `WHERE`, `ORDER BY`, and predict which rows they return.
5. Explain why a database is better than one big spreadsheet for shared data.

Explanation

What is a database?

A **database** is an organised collection of data stored so that it can be searched, updated and shared easily. A **database management system (DBMS)** is the software that stores the database and answers requests: for example MySQL, PostgreSQL, SQLite, Oracle and Microsoft SQL Server. Every time you log in to an app, check your grades online or search a shop's products, a DBMS is answering a request in the background.

Why not just one big spreadsheet?

Problem with one shared spreadsheet	How a database handles it
The same information is typed many times (a student's major on every course row), so copies disagree	Each fact is stored **once** and linked where needed
Two people editing at once overwrite each other	The DBMS manages many users at the same time safely
Anyone can type anything (e.g. "two" in a number column)	Each column has a **data type** and rules
Slow with millions of rows	Designed to search huge amounts of data quickly
Everyone sees everything	Different users get different permissions

Database models

A **database model** is the way the data is organised.

Model	Structure	Era / example
Hierarchical	A tree: each record has one parent (like folders inside folders)	1960s; IBM IMS
Network	Records linked in a web; a record may have several parents	Late 1960s–1970s; CODASYL systems
Relational	Data in **tables**, linked by shared values (keys)	Proposed by E. F. Codd in 1970; most databases today

Today the relational model dominates. (Newer "NoSQL" databases store data as documents or key–value pairs for special needs, but they are beyond this module.)

Tables, rows, columns, keys

In a **relational database**:

- A **table** stores data about one kind of thing (students, courses).
- A **row** (also called a **record**) is one item: one student.
- A **column** (also called a **field** or **attribute**) is one property: name, major.
- A **primary key** is a column whose value is **unique** for every row and never empty, so each row can be identified exactly. Names are poor keys (two students can share a name), so we use an ID.
- A **foreign key** is a column in one table that holds the primary key of a row in *another* table. It creates a **relationship** between the tables.

Students (primary key: `student_id`)

student_id	name	major	year	gpa
101	Anu	Computer Science	1	3.6
102	Bat	Economics	2	3.1
103	Chen	Computer Science	2	3.9
104	Dana	Design	1	3.4
105	Erdene	Economics	1	2.8
106	Fatima	Computer Science	3	3.2

Courses (primary key: `course_id`)

course_id	title	credits
CS101	Foundations of CS & AI	3
EC110	Intro to Economics	3
DS120	Visual Design	2

Enrolments (foreign keys: `student_id` → Students, `course_id` → Courses)

student_id	course_id
101	CS101

student_id	course_id
102	EC110
103	CS101
104	DS120
104	CS101
105	EC110
106	CS101

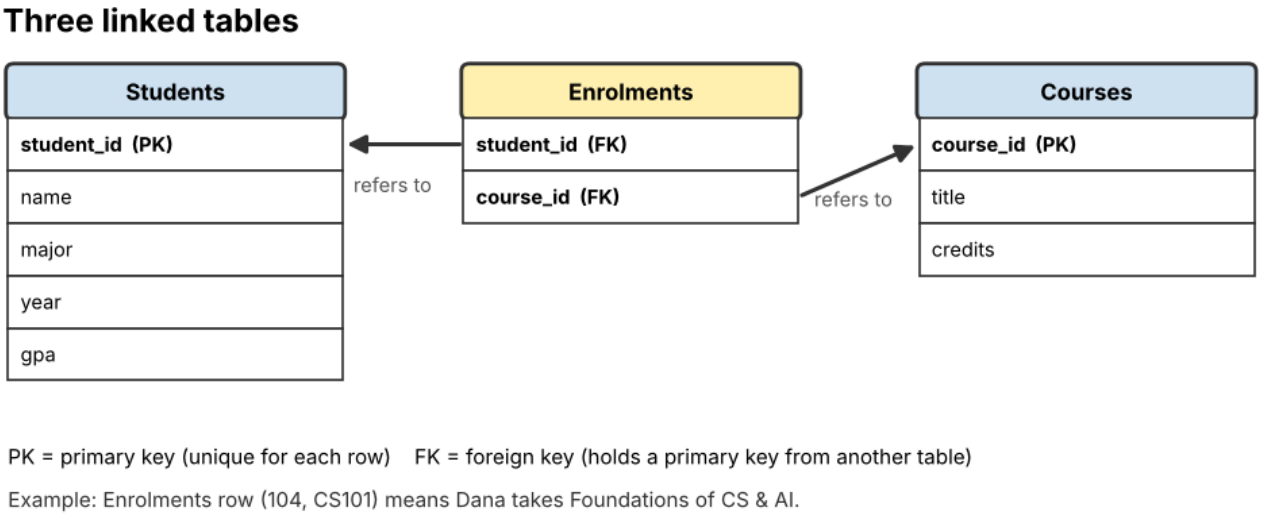


Figure 1. Enrolments links Students and Courses. Each student's name is stored only once, in Students; Enrolments stores only the ID.

SQL: asking the database questions

SQL (Structured Query Language, often said "sequel") is the standard language for relational databases. It is **declarative**: you describe *what* you want, and the DBMS works out *how* to find it (see Topic 02).

A basic **query** has this shape:

```
SELECT column1, column2     -- which columns to show
FROM   TableName            -- which table to look in
WHERE  condition            -- which rows to keep (optional)
ORDER BY column ASC|DESC;   -- how to sort the result (optional)
```

Part	Meaning
`SELECT name, gpa`	Show only these columns. `SELECT *` means all columns
`FROM Students`	Look in this table
`WHERE major = 'Economics'`	Keep only rows where the condition is true. Text goes in single quotes; numbers do not
Comparison	=, <> (not equal), <, >, <=, >=

Part	Meaning
Combine conditions	`AND` (both true), `OR` (at least one true), `NOT` — the same logic as Topic 08
`ORDER BY gpa DESC`	Sort the result; `ASC` = smallest first (the default), `DESC` = largest first

How to work out a query by hand: (1) go to the table in `FROM`; (2) go through it row by row and keep the rows where `WHERE` is true; (3) sort the kept rows by `ORDER BY`; (4) show only the columns in `SELECT`.

Worked Example: Find all Computer Science students

Question: Which students major in Computer Science, best GPA first?

```
SELECT name, gpa
FROM Students
WHERE major = 'Computer Science'
ORDER BY gpa DESC;
```

1. **FROM:** look at the Students table (6 rows).
2. **WHERE:** check each row. 101 Anu ✔, 102 Bat ✘ (Economics), 103 Chen ✔, 104 Dana ✘, 105 Erdene ✘, 106 Fatima ✔. Three rows kept.
3. **ORDER BY gpa DESC:** sort the kept rows from highest GPA: Chen 3.9, Anu 3.6, Fatima 3.2.
4. **SELECT:** show only name and gpa.

Result:

name	gpa
Chen	3.9
Anu	3.6
Fatima	3.2

Using a relationship (preview). "Who is enrolled in CS101?" needs two tables: find the IDs in Enrolments where `course_id = 'CS101'` (101, 103, 104, 106), then look up each ID in Students: **Anu, Chen, Dana, Fatima**. SQL does this with a `JOIN`, which is beyond the basic level of this module, but the idea (follow the key from one table to the other) is what matters.

Activity

Use the Students table above.

1. **Predict the rows.** Write the result of each query.
   - (a) `SELECT name FROM Students WHERE year = 1;`
   - (b) `SELECT name, major FROM Students WHERE gpa >= 3.5;`
   - (c) `SELECT name FROM Students WHERE major = 'Economics' AND year = 1;`
   - (d) `SELECT * FROM Students WHERE major = 'Design' OR gpa < 3.0;`
2. **Write the query.** Write SQL for: "Show the names and GPAs of **all** students, lowest GPA first."

3. **Find the error.** This query gives an error. Why? `SELECT name FROM Students WHERE major = Economics;`
4. **Keys.** (a) Why is `name` a poor choice of primary key for Students? (b) In Enrolments, student 104 appears twice. Is that a problem? (c) Using the Enrolments and Courses tables, which courses is Dana taking?

Answer

1. (a) **Anu, Dana, Erdene** (the three year-1 students). (b) **Anu – Computer Science; Chen – Computer Science** (3.6 and 3.9; Fatima's 3.2 is too low). (c) **Erdene** (Bat is Economics but year 2). (d) Two full rows: **104 Dana Design 1 3.4** (matches the first condition) and **105 Erdene Economics 1 2.8** (matches the second). With `OR`, one true condition is enough.
2. `SELECT name, gpa FROM Students ORDER BY gpa ASC;` → Erdene 2.8, Bat 3.1, Fatima 3.2, Dana 3.4, Anu 3.6, Chen 3.9. (`ASC` may be left out because it is the default.)
3. `Economics` is text, so it must be in single quotes: `'Economics'`. Without quotes the DBMS thinks it is the name of a column, and there is no such column.
4. (a) Two students can have the same name, so it would not identify one row exactly. A primary key must be unique. (b) No. `student_id` is a primary key in Students, but in Enrolments it is a **foreign key**, and one student can enrol in several courses. (c) Dana is 104 → Enrolments rows with 104 → DS120 and CS101 → **Visual Design** and **Foundations of CS & AI**.

Common Misunderstandings

- "*A database is just a big spreadsheet.*" Tables look similar, but a database adds keys, relationships, data types, safe multi-user access and fast searching.
- "*`WHERE` chooses columns.*" `WHERE` chooses **rows**; `SELECT` chooses **columns**.
- "*`AND` gives more results than `OR`.*" It is the opposite: `AND` needs both conditions, so it usually keeps fewer rows.

Quick Self-Check

1. What property must every primary key value have? *(It must be unique and not empty.)*
2. Which part of a query sorts the results? *(`ORDER BY`.)*
3. Who proposed the relational model, and when? *(E. F. Codd, 1970.)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** chapter "Databases and SQL"; predict-the-rows questions using the same Students table.
- **PPT Slides:** spreadsheet vs database table, Figure 1, the four-step "how to read a query" method animated on the worked example.
- **Video Tutorial:** not selected for video in Stage 2.

References

1. Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM, 13*(6), 377–387.
2. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson. (Database models, keys, SQL.)
3. SQLite. (2024). *SQL As Understood By SQLite: SELECT*. https://www.sqlite.org/lang_select.html (Used to run and check all queries.)
4. Foundations of CS & AI 2026, Chapter 3 course slides ("Databases: database models, relational databases, SQL").

The Students, Courses and Enrolments tables are the team's own invented data.

05 Algorithms

Chapter: Chapter 3: Software & Algorithms
Status: Drafted (V2.0, 3 Nov 2026). V1.0 covered searching only; V2.0 adds features and representation of algorithms, sorting, iteration and recursion, and efficiency. All traces were checked by running the code. Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: Topic 02 (variables, conditions, loops). A **list** is explained below in one line; Topic 06 covers it in depth.
Draft note: Prepared with Claude assistance (AI-use records AI-4 and AI-5). Every trace and count below was produced by running the code and compared with the hand trace.

Learning Objectives

After this topic, a beginner can:

1. Define **algorithm** and name its five features.
2. Represent an algorithm in **natural language**, **pseudocode** and a **flowchart**.
3. Carry out **linear search** and **binary search** by hand, and explain why binary search needs a **sorted** list.
4. Carry out **bubble sort** and **selection sort** by hand.
5. Explain the difference between **iteration** and **recursion** and trace a simple recursive function.
6. Compare algorithms by counting steps, and explain intuitively why some grow much faster than others.

Explanation

What is an algorithm?

An **algorithm** is a finite list of exact steps that solves a problem. A recipe is similar, but an algorithm must be clear enough that a computer could follow it with no guessing. Every program from Topic 02 carries out one or more algorithms.

A good algorithm has five **features** (Knuth's classic list):

Feature	Meaning	What breaks it
Finiteness	It always stops after a finite number of steps	"Keep adding 1 until the number is negative" never stops
Definiteness	Every step is exact, with only one meaning	"Add a little salt"
Input	It takes zero or more values to work on	–
Output	It produces at least one result	A procedure that never reports anything
Effectiveness	Every step is simple enough to actually be done	"Write down every digit of π"

Three ways to write an algorithm

Problem: find whether a target value is in a list.

1. Natural language: Look at the items one by one from the start. If an item equals the target, say where it is and stop. If you reach the end, say "not found".

2. Pseudocode (structured, program-like English that is not tied to one language):

```
for each position i from 0 to the last position
    if item[i] = target then
        return i
return "not found"
```

3. Flowchart: a diagram with standard shapes: an oval for **start/stop**, a rectangle for a **process** step, a diamond for a **decision** (yes/no question), and arrows for the order.

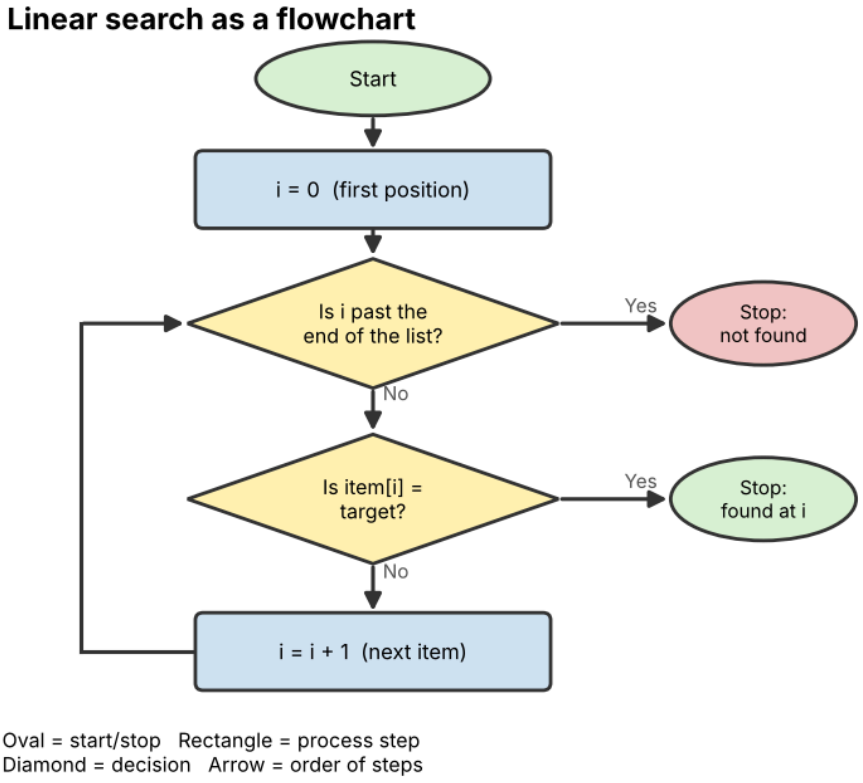


Figure 1. Linear search as a flowchart. The loop is the arrow going back up from "move to the next item".

Searching

A **list** is an ordered collection of items; each position has a number called an **index**, starting at 0. **Searching** means finding where a target value is in a list (or finding that it is not there).

- **Linear search:** check items one by one from the start until you find the target or reach the end. Works on any list.
- **Binary search:** works only on a **sorted** list. Look at the middle item. If it equals the target, stop. If the target is bigger, discard the left half; if smaller, discard the right half. Repeat on the half that is left. Each step halves the search area.

For binary search, keep two markers, **low** and **high**, for the part still being searched, and compute **mid** = (low + high) // 2 (round down). Stop with "not found" when low > high.

```
def binary_search(items, target):
    low, high = 0, len(items) - 1
    while low <= high:
        mid = (low + high) // 2
        if items[mid] == target:
            return mid          # found: return the index
        elif items[mid] < target:
            low = mid + 1       # search the right half
        else:
            high = mid - 1      # search the left half
    return -1                   # not found
```

Sorting

Sorting means putting items in order (smallest to largest, A to Z). Sorting matters because sorted data is much faster to search (binary search) and easier for people to read.

Bubble sort. Go through the list comparing each **neighbouring pair**. If they are in the wrong order, **swap** them. After one full **pass**, the largest value has "bubbled" to the end. Repeat passes on the unsorted part. Stop when a pass makes no swaps.

Selection sort. Find the **smallest** item in the unsorted part and swap it into the first unsorted position. Now one more item is in its final place. Repeat.

```
def bubble_sort(items):
    n = len(items)
    for p in range(n - 1):                # one pass each time
        swapped = False
        for i in range(n - 1 - p):        # the sorted end is skipped
            if items[i] > items[i + 1]:
                items[i], items[i + 1] = items[i + 1], items[i]   # swap
                swapped = True
        if not swapped:
            break                         # no swaps: already sorted
    return items
```

Iteration and recursion

There are two ways to make an algorithm repeat work:

- **Iteration:** use a loop (`for`, `while`) to repeat steps. All the algorithms above are iterative.
- **Recursion:** a function **calls itself** on a smaller version of the same problem. Every recursive function needs:
    1. a **base case**, a simple case answered directly, which stops the recursion; and

2. a **recursive case**, which makes the problem smaller and calls the function again.

Example: the **factorial** of n, written n!, is n × (n−1) × … × 1. So 4! = 4 × 3 × 2 × 1 = 24. Notice that 4! = 4 × 3!, so the problem contains a smaller copy of itself.

```
def factorial(n):          # recursive version
    if n <= 1:             # base case
        return 1
    return n * factorial(n - 1)   # recursive case: smaller problem

def factorial_loop(n):     # iterative version, same result
    result = 1
    for k in range(2, n + 1):
        result = result * k
    return result
```

Without a base case, a recursive function would call itself forever (in practice the program crashes with a "maximum recursion depth" error). The computer keeps track of the unfinished calls on a **call stack**, a stack like the one in Topic 06: the last call started is the first to finish.

Efficiency: how fast does the work grow?

We compare algorithms by counting their basic steps (such as comparisons) as the input gets bigger, not by timing them on one computer.

List size *n*	Linear search, worst case (checks)	Binary search, worst case (checks)
8	8	4
1,000	1,000	10
1,000,000	1,000,000	20
1,000,000,000	1,000,000,000	30

Linear search grows in step with *n*: double the list, double the work. Binary search grows by only **one** check each time the list doubles, because each check halves what is left. Bubble and selection sort compare roughly every pair of items, so for *n* items they need about $n^2/2$ comparisons: 10 items → 45 comparisons, 1,000 items → about 500,000. That is why real systems use faster sorting methods (such as merge sort) for big data.

(Computer scientists write these growth rates in "Big O" notation: linear search is O(n), binary search O(log n), bubble sort $O(n^2)$. The notation is optional in this module; the idea of how the work grows is what matters.)

Worked Example 1: Binary search

List (sorted): `[3, 8, 12, 19, 25, 31, 40, 47]` with indexes 0 to 7. **Target: 31.**

Step	low	high	mid	items[mid]	Decision
1	0	7	3	19	19 < 31, so discard the left half: low = 4
2	4	7	5	31	Found at index 5

Binary search needed **2 checks**. Linear search would check 3, 8, 12, 19, 25, 31, which is **6 checks**.

Worked Example 2: Bubble sort

Sort `[5, 2, 9, 1, 6]`.

Pass 1 (compare each neighbouring pair):

Compare	Swap?	List after
5 and 2	Yes	[2, 5, 9, 1, 6]
5 and 9	No	[2, 5, 9, 1, 6]
9 and 1	Yes	[2, 5, 1, 9, 6]
9 and 6	Yes	[2, 5, 1, 6, **9**]

9, the largest, is now in its final place.

Pass	List after the pass	Swaps
1	[2, 5, 1, 6, **9**]	3
2	[2, 1, 5, **6, 9**]	1
3	[1, 2, **5, 6, 9**]	1
4	[1, 2, 5, 6, 9]	0 → stop, sorted

Worked Example 3: Tracing recursion

`factorial(4)`:

```
factorial(4) = 4 * factorial(3)          waits
    factorial(3) = 3 * factorial(2)      waits
        factorial(2) = 2 * factorial(1)  waits
            factorial(1) = 1             base case reached
        factorial(2) = 2 * 1 = 2         returns 2
    factorial(3) = 3 * 2 = 6             returns 6
factorial(4) = 4 * 6 = 24                returns 24
```

The calls go **down** until the base case, then the answers come back **up** in reverse order (last call started, first call finished).

Activity

1. **Binary search.** Using `[3, 8, 12, 19, 25, 31, 40, 47]`: (a) search for **8**; fill in a table like Worked Example 1. How many checks? (b) Search for **10**. What happens, and how many checks before you can say "not found"?

2. **Why sorted?** Why would binary search give wrong results on `[12, 3, 47, 8]`?

3. **Bubble sort.** Sort `[4, 3, 1, 2]` with bubble sort. Write the list after each pass. How many passes until a pass makes no swaps?

4. **Selection sort.** Sort `[29, 10, 14, 37, 13]` with selection sort. Write the list after each step.

5. **Recursion.** (a) Trace `factorial(5)`. (b) This function is meant to add 1 + 2 + ... + n. What does `sum_to(4)` return? (c) What would happen if the line `if n == 0: return 0` were deleted?

```
def sum_to(n):
    if n == 0:
        return 0
    return n + sum_to(n - 1)
```

6. **Compare.** A phone contact list has 1,000,000 names in alphabetical order. At most how many checks does each search method need to find one name? Which would you choose and why?

7. **Spot the broken feature.** Which algorithm feature does each break? (a) "Pick a good number." (b) "Start at 1 and keep doubling until you reach 0."

Answer

1. (a) Step 1: low 0, high 7, mid 3, value 19 > 8, so high = 2. Step 2: low 0, high 2, mid 1, value 8. Found at index 1 after **2 checks**. (b) Step 1: mid 3 (19) > 10, high = 2. Step 2: low 0, high 2, mid 1 (8) < 10, low = 2. Step 3: low 2, high 2, mid 2 (12) > 10, high = 1. Now low (2) > high (1), so **not found after 3 checks**.
2. The method assumes that everything left of the middle is smaller and everything right is bigger. In an unsorted list that is false, so discarding half the list can throw away the target.
3. Pass 1: [3, 1, 2, **4**]. Pass 2: [1, 2, **3, 4**]. Pass 3: [1, 2, 3, 4] with **no swaps**, so stop. **Three passes** (the third pass confirms the list is sorted).
4. Step 1: smallest is 10, swap with 29 → [10, 29, 14, 37, 13]. Step 2: smallest of the rest is 13, swap with 29 → [10, 13, 14, 37, 29]. Step 3: smallest of the rest is 14, already in place → [10, 13, 14, 37, 29]. Step 4: smallest of the rest is 29, swap with 37 → [10, 13, 14, 29, 37]. Sorted.
5. (a) factorial(5) = 5 × factorial(4) = 5 × 4 × factorial(3) = ... = 5 × 4 × 3 × 2 × 1 = **120**. (b) sum_to(4) = 4 + sum_to(3) = 4 + 3 + sum_to(2) = 4 + 3 + 2 + sum_to(1) = 4 + 3 + 2 + 1 + sum_to(0) = 4 + 3 + 2 + 1 + 0 = **10**. (c) There would be **no base case**: sum_to(0) would call sum_to(−1), then sum_to(−2), forever, until the program crashes with a recursion-depth error. This breaks *finiteness*.
6. Linear search: up to **1,000,000** checks. Binary search: at most **20** checks. Choose binary search, because the list is already sorted.
7. (a) **Definiteness**: "good" is not exact. (b) **Finiteness**: doubling a positive number never reaches 0, so it never stops.

Common Misunderstandings

- "*Binary search is always better.*" It needs a sorted list. If the list is unsorted and you search only once, sorting first costs more than one linear search.
- "*Recursion is a loop with a different name.*" Recursion solves a problem by solving smaller copies of the same problem; it needs a base case, and it uses the call stack.
- "*A faster computer fixes a slow algorithm.*" For large inputs, a better algorithm (20 checks instead of 1,000,000) beats a faster machine.

Quick Self-Check

1. Which flowchart shape is used for a yes/no decision? *(Diamond.)*
2. After the first pass of bubble sort, which item is guaranteed to be in its final place? *(The largest.)*
3. What are the two parts every recursive function needs? *(A base case and a recursive case.)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** chapter "Algorithms"; tracing questions for binary search, bubble sort, selection sort and recursion; growth table for comparison questions.
- **PPT Slides:** features table, Figure 1 flowchart, binary-search table step by step, bubble-sort pass table, recursion call trace.
- **Video Tutorial:** "Binary search by hand" (planned tutorial 2), using Worked Example 1 and Activity 1. Bubble sort is a candidate for a second short clip.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. (Binary search, sorting, recursion, growth of functions.)
2. Knuth, D. E. (1997). *The Art of Computer Programming, Vol. 1: Fundamental Algorithms* (3rd ed.). Addison-Wesley. (The five features of an algorithm.)
3. Bhargava, A. Y. (2016). *Grokking Algorithms*. Manning. (Beginner-level explanations of binary search, selection sort, recursion and the call stack.)
4. Foundations of CS & AI 2026, Chapter 3 course slides ("Algorithms: features, representation, basic algorithms (sorting, searching), iteration and recursion").

The lists, targets and traces are the team's own examples.

06 Data Structures

Chapter: Chapter 3: Software & Algorithms
Status: Drafted (V2.0, 3 Nov 2026). Stack, queue and tree traces checked by running code. Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: Topic 05 (lists and indexes; binary search).
Draft note: Prepared with Claude assistance (AI-use record AI-5). Team members must review and be able to explain every part.

Learning Objectives

After this topic, a beginner can:

1. Explain why the way data is organised affects how fast and how easily we can use it.
2. Describe an **array**, a **record**, a **linked list**, a **stack**, a **queue**, a **tree**, a **binary search tree** and a **graph**.
3. Trace **push/pop** on a stack and **enqueue/dequeue** on a queue.
4. Search a small binary search tree by hand.
5. Choose a suitable structure for a simple task and justify the choice.

Explanation

Why structure matters

A **data structure** is a way of organising data in memory so that it can be used efficiently. Think of a library: books thrown in a pile, books on shelves by number, and a card catalogue all hold the same books, but finding one takes very different amounts of time. Choosing the structure is choosing which operations (finding, adding, removing) will be fast.

Data structures and algorithms go together: binary search (Topic 05) only works because the data is stored as a *sorted array*.

The main structures

Structure	Picture	Key idea	Good at	Weak at
Array	A row of numbered boxes	Fixed-size sequence of items of the same type, each reached by its **index** (`a[3]`)	Jumping straight to any position	Inserting in the middle (everything after must shift)
List (dynamic array, e.g. Python list)	Array that can grow	Like an array but can change size	Same as array; easy to add at the end	Inserting at the front of a long list
Record (struct)	A form with labelled boxes	Groups **different kinds** of data about one thing under field names	Keeping related data together: `{name: "Anu", age: 19, gpa: 3.6}`	Not a collection by itself (use an array of records)

Structure	Picture	Key idea	Good at	Weak at
Linked list	A chain of boxes, each pointing to the next	Each **node** holds a value and a **pointer** (link) to the next node	Inserting or removing anywhere by changing links	Finding the 500th item (must walk from the start)
Stack	A pile of plates	**LIFO**: Last In, First Out. Add (**push**) and remove (**pop**) only at the **top**	Undo, Back button, matching brackets, recursion (Topic 05)	Reaching items at the bottom
Queue	A line at a canteen	**FIFO**: First In, First Out. Add (**enqueue**) at the **back**, remove (**dequeue**) from the **front**	Fair waiting lines: print jobs, messages, CPU ready queue (Topic 01)	Reaching items in the middle
Tree	A family tree / folder tree	Nodes linked **parent → children**, starting from one **root**; no loops	Hierarchies: folders, organisation charts	–
Binary search tree (BST)	A tree where each node has at most 2 children	Left child smaller, right child bigger	Fast searching, adding and removing in sorted data	Gets slow if it becomes very lopsided
Graph	A map of cities and roads	**Nodes** (vertices) joined by **edges**; any node can link to any other; loops allowed	Networks: maps, friendships, web links	–

The record, array and linked list are *how* data is laid out; the stack and queue are defined by *rules* for adding and removing; trees and graphs model *relationships*.

Linked list

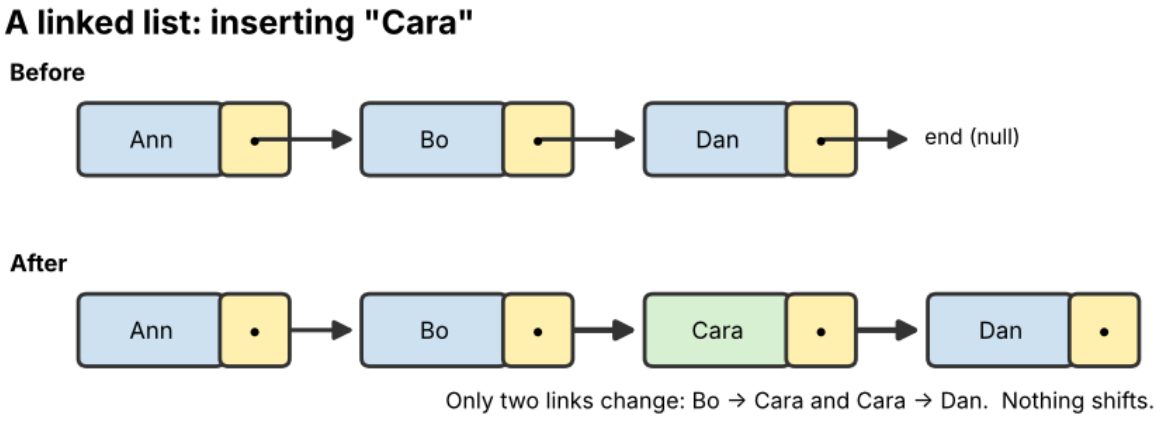


Figure 1. Each node stores a value and a pointer to the next node. To insert "Cara" between "Bo" and "Dan", only two links change: Bo now points to Cara, and Cara points to Dan. Nothing needs to shift.

Stack and queue

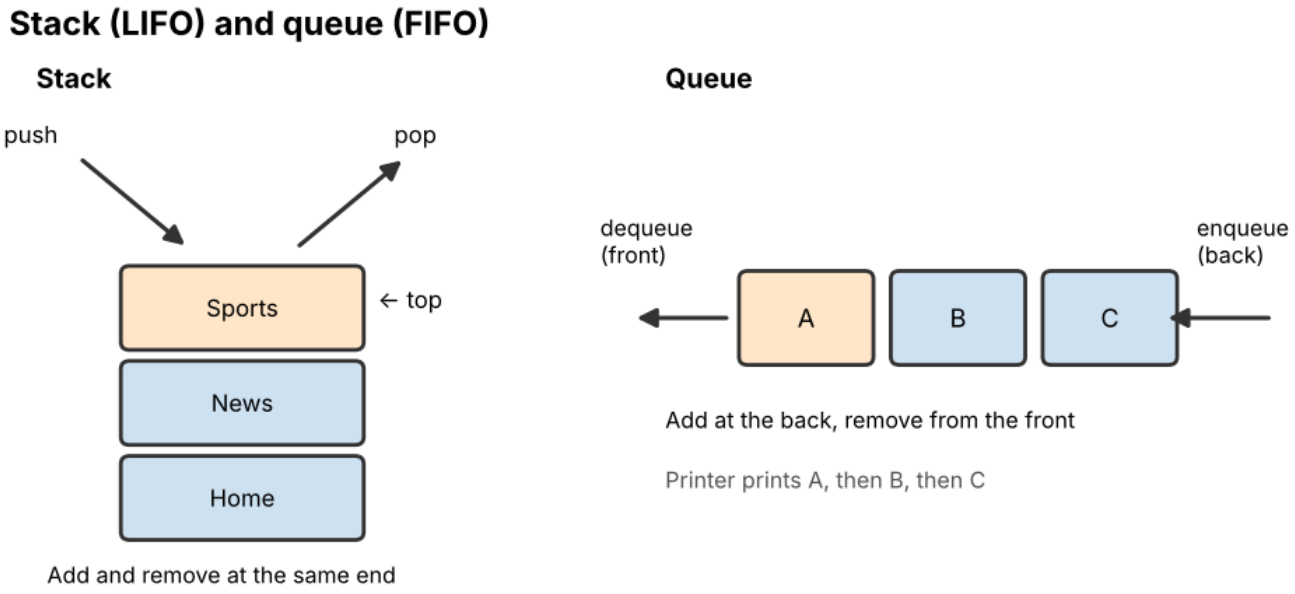


Figure 2. A stack adds and removes at the same end (the top). A queue adds at the back and removes from the front.

Trees and binary search trees

Tree words: the top node is the **root**; a node directly below another is its **child**, and the one above is its **parent**; nodes with no children are **leaves**. Folders on your computer form a tree (Topic 01).

A **binary tree** is a tree where every node has **at most two** children, called left and right. A **binary search tree** adds one rule: for every node, all values in its **left** subtree are **smaller** and all values in its **right** subtree are **bigger**. To search, start at the root and go left or right at each step, just like binary search halves a list.

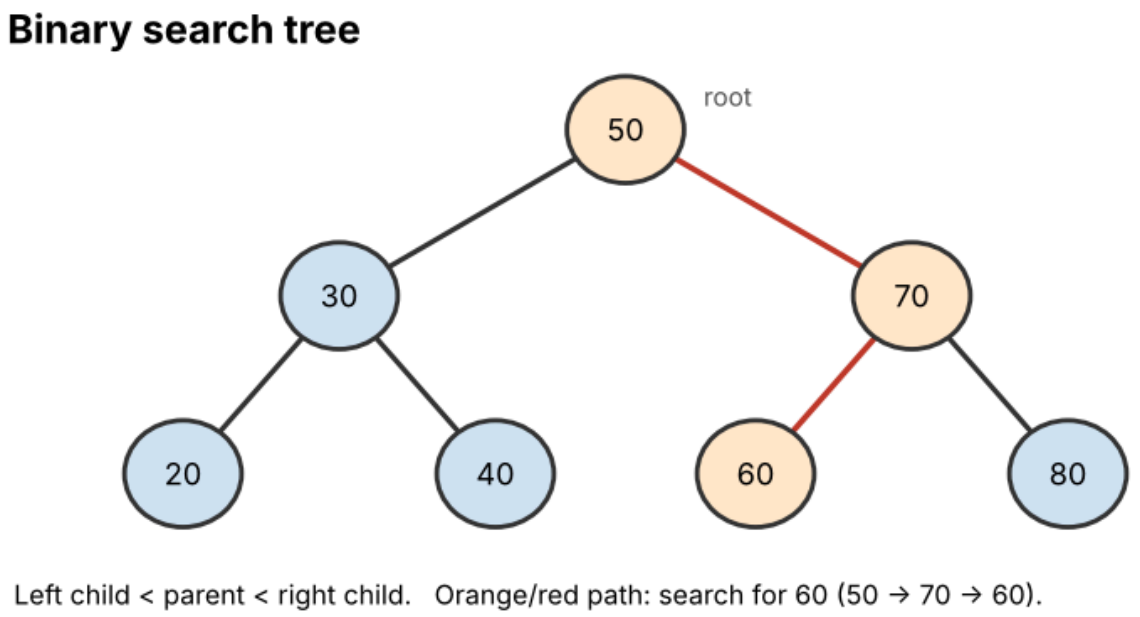


Figure 3. A binary search tree built by inserting 50, 30, 70, 20, 40, 60, 80 in that order. Searching for 60: 60 > 50 → go right to 70; 60 < 70 → go left to 60. Found in 3 checks.

Graphs

A **graph** is the most general structure: nodes connected by edges, with no parent/child rule. Edges may have a direction (one-way streets, "follows" on social media) and a **weight** (distance, cost). Searching through graphs is the heart of Topic 11 (planning and search), and the semantic networks of Topic 09 are graphs with labelled edges.

Worked Example: The browser Back button is a stack

The browser keeps the pages you visit on a stack. The page on **top** is the one you are looking at.

Action	Operation	Stack (bottom → top)	Page shown
Open Home	push Home	[Home]	Home
Click News	push News	[Home, News]	News
Click Sports	push Sports	[Home, News, Sports]	Sports
Press Back	pop (removes Sports)	[Home, News]	News
Click Weather	push Weather	[Home, News, Weather]	Weather
Press Back	pop (removes Weather)	[Home, News]	News
Press Back	pop (removes News)	[Home]	Home

Back always returns to the **most recently** visited page that is still on the stack: last in, first out. Notice that after pressing Back from Sports and then visiting Weather, Sports is gone. That matches how real browsers behave.

Compare with a queue. Three documents are sent to one printer: A, then B, then C. The printer queue is [A, B, C] (front → back). It prints A first (dequeue), then B, then C: first in, first out. A stack here would print C first, which would be unfair to the person who sent A.

Activity

1. **Trace a stack.** Start with an empty stack. Carry out: push 5, push 8, pop, push 2, push 7, pop, pop. What is on the stack at the end, and which values were popped, in order?
2. **Trace a queue.** Start with an empty queue. Carry out: enqueue Anu, enqueue Bat, dequeue, enqueue Chen, enqueue Dana, dequeue. Who is still waiting, in order from the front?
3. **Search a BST.** Using Figure 3: (a) list the nodes visited when searching for 40; (b) list the nodes visited when searching for 65, and say how you know it is not there.
4. **Insert into a BST.** Insert 35 into the tree in Figure 3. Where does it go?
5. **Choose a structure.** Choose the best structure for each task and give one reason: (a) the Undo history in a text editor; (b) jobs waiting for a shared printer; (c) a family tree; (d) a shopping list you read from top to bottom; (e) one student's name, age and major kept together; (f) cities and the roads between them, for a route-planning app.
6. **Find the error.** A student says: "A queue is a good structure for the Back button, because pages are stored in the order they were visited." What is wrong with this?

Answer

1. push 5 → [5]; push 8 → [5, 8]; pop → removes **8** → [5]; push 2 → [5, 2]; push 7 → [5, 2, 7]; pop → removes **7** → [5, 2]; pop → removes **2** → [5]. **Stack at the end: [5]. Popped in order: 8, 7, 2.**
2. enqueue Anu → [Anu]; enqueue Bat → [Anu, Bat]; dequeue → Anu leaves → [Bat]; enqueue Chen → [Bat, Chen]; enqueue Dana → [Bat, Chen, Dana]; dequeue → Bat leaves → [Chen, Dana]. **Still waiting: Chen, then Dana.**
3. (a) 50 → (40 < 50, go left) 30 → (40 > 30, go right) **40**. Found after 3 checks. (b) 50 → (65 > 50, right) 70 → (65 < 70, left) 60 → (65 > 60, go right) — 60 has **no right child**, so 65 is **not in the tree**. We know because a BST would have to store 65 exactly there.
4. 35 < 50 → left to 30; 35 > 30 → right to 40; 35 < 40 → left of 40, which is empty. **35 becomes the left child of 40.**
5. (a) **Stack**: Undo reverses the most recent change first (LIFO). (b) **Queue**: jobs are printed in the order they arrived (FIFO), which is fair. (c) **Tree**: each person has parents above and children below; it is a hierarchy. (Strictly, real family trees with two parents per child are graphs, but a tree is the usual simple model.) (d) **List/array**: an ordered sequence read from start to end. (e) **Record**: different kinds of data (text, number) about one thing under field names. (f) **Graph**: any city can connect to many others, and roads can form loops.
6. A queue would remove the **oldest** page first, so pressing Back on the Sports page would jump to Home, the first page visited. Back must return the **most recent** previous page, which needs LIFO, i.e. a **stack**. Both store pages in visiting order; the difference is **which end you remove from**.

Common Misunderstandings

- "*Stacks and queues store data differently.*" Both can be built from the same array or linked list. What differs is the **rule** for removing items.
- "*A tree in computing grows upward.*" By convention the root is drawn at the **top** and the tree grows downward.
- "*Arrays and linked lists are equally fast for everything.*" Arrays jump straight to any index; linked lists must walk link by link, but they insert and remove without shifting.

Quick Self-Check

1. Which structure is LIFO? Which is FIFO? *(Stack; queue.)*
2. In a BST, where are values smaller than a node stored? *(In its left subtree.)*
3. Which structure groups different kinds of data about one thing under field names? *(A record.)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** chapter "Data Structures"; stack/queue trace questions, BST search, and "choose a structure" questions.
- **PPT Slides:** the structures table split over several slides, Figures 1–3, the Back-button table animated row by row.
- **Video Tutorial:** not selected as a separate tutorial in Stage 2; the Back-button stack may appear as a short clip in the algorithms tutorial.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. (Stacks, queues, linked lists, binary search trees, graphs.)
2. Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). *Data Structures and Algorithms in Python*. Wiley. (Beginner-friendly treatment of each structure in Python.)
3. Foundations of CS & AI 2026, Chapter 3 course slides ("Data structures: arrays, records, linked list, stacks, queues, trees, binary trees, graphs").

The browser, printer and tree examples are the team's own.

07 Intro to Symbolic AI

Chapter: Chapter 4: Symbolic AI
Status: Drafted (V2.0, 3 Nov 2026). Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: None required. Topics 02 (conditions) and 06 (trees) help.
Draft note: Prepared with Claude assistance (AI-use record AI-5). Historical dates were checked against the references below. Team members must review and be able to explain every part.

Learning Objectives

After this topic, a beginner can:

1. Explain what **artificial intelligence (AI)** and **Symbolic AI** are.
2. Describe the main idea that thinking can be done by **manipulating symbols with rules**.
3. Compare Symbolic AI with **data-driven AI** (machine learning) and say where each works well.
4. Give at least two historical examples of Symbolic AI and place them on a timeline.
5. Follow a small set of rules to reach a conclusion, as a Symbolic AI program does.

Explanation

What is AI?

Artificial intelligence (AI) is the field of building computer systems that do tasks which would need intelligence if a person did them: reasoning, planning, understanding language, recognising objects, making decisions.

There are two big families of approaches:

- **Symbolic AI**: people write knowledge down explicitly as **symbols** (words, names, numbers) and **rules**, and the program **reasons** with them.
- **Data-driven AI** (also called **machine learning**): the program is given many examples and **learns patterns** from them by adjusting numbers inside a model; nobody writes the rules by hand.

The core idea of Symbolic AI

A **symbol** is something that stands for something else: the word `cat` stands for a real cat; `raining` stands for the fact that it is raining. Symbolic AI rests on an idea stated by Allen Newell and Herbert Simon, the **physical symbol system hypothesis** (1976): a system that can store symbols and change them by rules has what it needs to act intelligently.

So a Symbolic AI system has three ingredients:

1. **Symbols** that represent things in the world: `penguin`, `has_feathers`, `can_fly`.

2. **Knowledge** written using those symbols: facts (`Tweety is a penguin`) and rules (`IF X is a bird THEN X has feathers`).
3. A **reasoning procedure** that applies the rules to the facts to produce new facts or decisions.

This is why Symbolic AI is sometimes called **rule-based AI**, **logic-based AI** or **GOFAI** ("Good Old-Fashioned AI", a nickname from the philosopher John Haugeland, 1985). The rest of Chapter 4 looks at each ingredient in turn:

Topic	Question it answers
08 Logic and Reasoning	How do we reason correctly with true/false statements?
09 Knowledge Representation	How do we store knowledge as symbols?
10 Rule-Based and Expert Systems	How does a program apply rules to facts to reach a conclusion?
11 Planning and Search	How does a program find a sequence of actions to reach a goal?
12 Symbolic NLP	How can grammar rules help a program handle language?

Symbolic AI vs data-driven AI

	Symbolic AI	Data-driven AI (machine learning)
Where the knowledge comes from	Written by human experts as rules	Learned automatically from many examples
How knowledge is stored	Readable symbols and rules	Millions of numbers (weights) inside a model
Can it explain its answer?	Yes: it can list the rules it used	Usually hard to explain
Needs lots of data?	No	Yes
Handles messy, uncertain input (photos, speech, handwriting)?	Poorly	Well
Changing its behaviour	Edit or add a rule	Retrain on new data
Typical uses	Tax and legal rule checking, configuration, planning, games with clear rules, route finding	Image recognition, speech recognition, translation, chatbots like ChatGPT

Many modern systems are **hybrid (mixed)**: for example, AlphaGo (2016) combined neural networks that learned from data with a symbolic **tree search** over possible moves.

A short history

Year	Event
1956	The **Dartmouth workshop** gives the field the name "artificial intelligence" (the term was coined by John McCarthy in the 1955 proposal).
1956	**Logic Theorist** (Newell, Shaw and Simon) proves theorems from *Principia Mathematica*, often called the first AI program.
1957–1959	**General Problem Solver** (Newell and Simon) tries to solve any problem stated as goals and rules.

Year	Event
1966	**ELIZA** (Joseph Weizenbaum) chats using keyword rules (see Topic 12).
1965 onward	**DENDRAL** (Stanford) identifies chemical molecules from measurements: the first expert system.
1972	**Prolog**, a programming language based on logic (see Topic 02).
1970s	**MYCIN** suggests treatments for blood infections with several hundred rules (see Topic 10).
1980s	Expert systems become a business, e.g. **XCON/R1** at Digital Equipment Corporation.
Late 1980s	An "**AI winter**": expert systems prove expensive to build and maintain, and funding falls.
1997	IBM's **Deep Blue** beats world chess champion Garry Kasparov, using massive search plus a hand-built evaluation of positions.
2010s onward	Machine learning dominates, but symbolic methods remain in route planning, scheduling, verification and hybrid systems.

Worked Example: An animal-guessing program

The program knows four animals: **eagle, penguin, dog, shark**. A person thinks of one, and the program asks yes/no questions. Its knowledge is four IF-THEN rules:

Rule	IF	THEN
R1	has feathers AND can fly	it is an **eagle**
R2	has feathers AND cannot fly	it is a **penguin**
R3	has fur	it is a **dog**
R4	has fins	it is a **shark**

The program asks questions in a fixed order, which forms a **decision tree** (a tree from Topic 06):

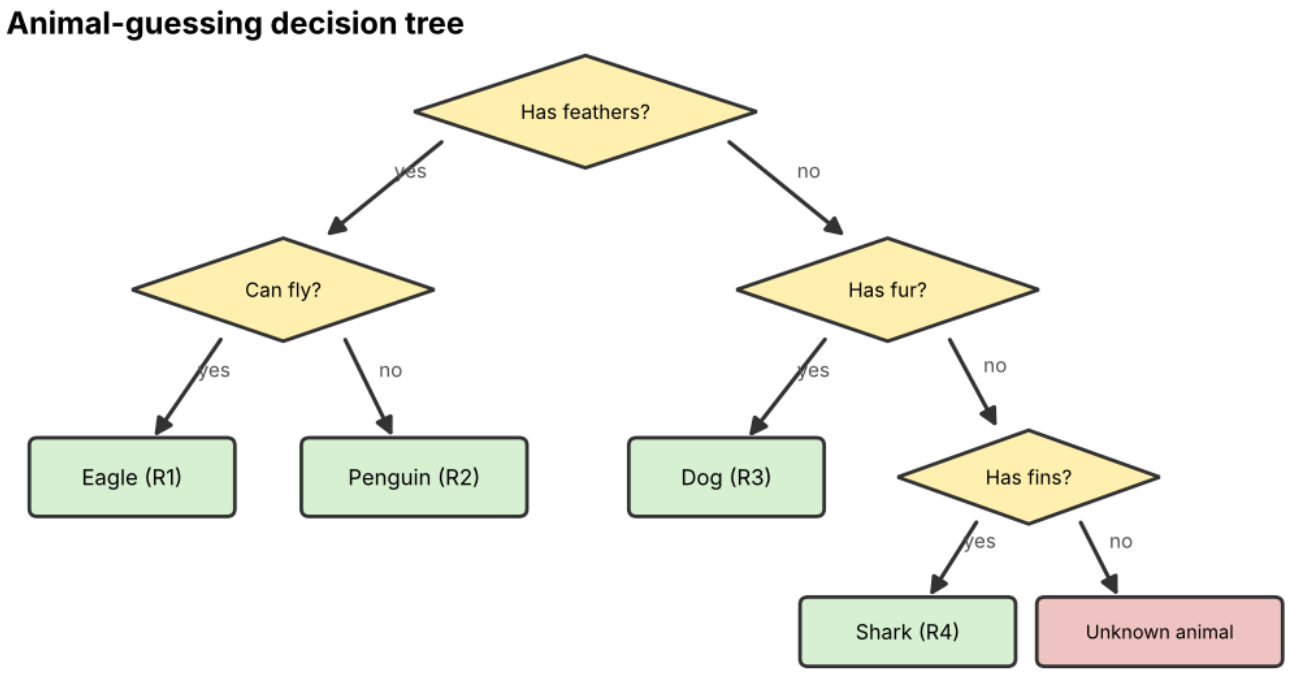


Figure 1. Each diamond is a yes/no question. Each leaf is a conclusion produced by one rule.

Run: the person is thinking of a penguin.

1. Program: "Does it have feathers?" Person: **yes**. New fact: *has feathers*.
2. Program: "Can it fly?" Person: **no**. New fact: *cannot fly*.
3. R2's conditions (has feathers AND cannot fly) are both true facts → R2 **fires**.
4. Program: "**It is a penguin.**" If asked *why*, it can answer: "Because it has feathers and cannot fly (rule R2)."

Three things to notice: every step is a readable symbol; the program can **explain** its answer; and it knows **nothing** outside its four rules. If the person thinks of a frog, the answers are no feathers, no fur, no fins, and no rule fires, so the program cannot answer. That brittleness is a typical limit of Symbolic AI.

Activity

1. **Run the program.** The person thinks of a **shark**. Write the questions asked, the answers, the facts added, and the rule that fires.
2. **Extend the knowledge.** Add the **frog** to the program. Write one new rule (R5) using a new symbol, and say where in Figure 1 a new question would go.
3. **Classify.** Label each system **S** (mainly symbolic), **D** (mainly data-driven) or **M** (mixed), and give a reason:
   - (a) A tax website that applies the official tax rules to your income.
   - (b) A spam filter trained on 100,000 emails that users marked as spam or not spam.
   - (c) A GPS app that finds the shortest route on a road map.
   - (d) Face unlock on a phone.
   - (e) AlphaGo.
   - (f) MYCIN.
4. **Explain a decision.** A hospital wants a system that recommends a drug dose, and doctors must be able to check *why* each recommendation was made. A bank wants a system that recognises handwritten cheque amounts. Which approach suits each better, and why?

Answer

1. "Does it have feathers?" **no** → fact: no feathers. "Does it have fur?" **no** → fact: no fur. "Does it have fins?" **yes** → fact: has fins. **R4 fires**: it is a shark.
2. Example: **R5: IF has no feathers AND has no fur AND has no fins AND lives on land and in water THEN it is a frog.** A new question "Does it live on land and in water?" goes where "Does it have fins?" is answered **no** (where the tree currently says "unknown animal"). Any rule that separates the frog from the other four animals using checkable facts is acceptable.
3. (a) **S**: the tax rules are written by people and applied exactly. (b) **D**: it learns what spam looks like from labelled examples. (c) **S**: it searches a map graph with an exact procedure (Topic 11). (d) **D**: it learns to recognise faces from many images. (e) **M**: learned neural networks plus symbolic tree search. (f) **S**: hand-written IF-THEN rules from experts.

4. **Hospital → Symbolic AI** (or at least a symbolic part): every recommendation can be traced to rules that doctors can read and check. **Bank → data-driven AI**: handwriting varies endlessly and is hard to describe in rules, but there are many examples to learn from.

Common Misunderstandings

- "*Symbolic AI is outdated and no longer used.*" Rule engines, planners, route finders, and logic checkers are used every day, often inside hybrid systems.
- "*Symbolic AI programs learn from experience.*" Classic symbolic systems do not learn; they only know what people wrote. Learning from data is the strength of machine learning.
- "*If a program uses IF statements, it is Symbolic AI.*" Ordinary programs use IF statements too. Symbolic AI is about representing **knowledge** as symbols and rules and having a general **reasoning procedure** use them.

Quick Self-Check

1. Name the three ingredients of a Symbolic AI system. *(Symbols, knowledge (facts and rules), a reasoning procedure.)*
2. Which approach can usually explain its answer by listing rules? *(Symbolic AI.)*
3. Which 1956 event named the field of AI? *(The Dartmouth workshop.)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** opening chapter of Part 2; classification questions (S/D/M) and timeline questions.
- **PPT Slides:** symbolic vs data-driven table, timeline, Figure 1 decision tree.
- **Video Tutorial:** not selected as a separate video; the animal-guessing run may open the expert-systems tutorial.

References

1. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Chapter 1: history of AI, Dartmouth, Logic Theorist, GPS, expert systems, AI winter.)
2. Newell, A., & Simon, H. A. (1976). Computer science as empirical inquiry: Symbols and search. *Communications of the ACM, 19*(3), 113–126. (Physical symbol system hypothesis.)
3. Haugeland, J. (1985). *Artificial Intelligence: The Very Idea*. MIT Press. (The term "GOFAI".)
4. Campbell, M., Hoane, A. J., & Hsu, F. (2002). Deep Blue. *Artificial Intelligence, 134*(1–2), 57–83.
5. Silver, D., et al. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature, 529*, 484–489. (AlphaGo as a hybrid system.)
6. Foundations of CS & AI 2026, Chapter 4 course slides.

The animal-guessing rules and the activity systems are the team's own examples.

08 Logic and Reasoning

Chapter: Chapter 4: Symbolic AI
Status: Drafted (V2.0, 3 Nov 2026; V1.0 24 Oct). V2.0 adds modus tollens, variables in rules, misunderstandings, self-check and reuse notes. Truth tables checked by a short program. Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: Topic 07 (what Symbolic AI is). No mathematics beyond everyday true/false thinking.
Draft note: Prepared with Claude assistance (AI-use records AI-4 and AI-5); to be checked by a team member.

Learning Objectives

1. Explain what a **proposition** is and combine propositions with AND, OR, NOT and IF-THEN.
2. Complete a truth table for a simple statement.
3. Use **modus ponens** and **modus tollens** to draw valid conclusions, and spot the two common invalid patterns.
4. Read a rule with a **variable** (X) and apply it to a specific case.

Explanation

Logic is the study of correct reasoning. Symbolic AI uses logic so that a computer can reason with statements in a fixed, checkable way.

A **proposition** is a statement that is either true or false, for example "It is raining". Questions and commands are not propositions. We usually name propositions with letters, such as **P** = "It is raining" and **Q** = "The road is wet".

Propositions can be combined:

Symbol	Name	Meaning	Example
P AND Q	Conjunction	True only if both are true	It is raining AND the road is wet
P OR Q	Disjunction	True if at least one is true	It is raining OR the sprinkler is on
NOT P	Negation	Flips true and false	It is NOT raining
IF P THEN Q	Implication	A rule: false only when P is true and Q is false	IF it is raining THEN the road is wet

A **truth table** lists every possible combination of true (T) and false (F) and gives the result:

P	Q	P AND Q	P OR Q	NOT P	IF P THEN Q
T	T	T	T	F	T
T	F	F	T	F	**F**

P	Q	P AND Q	P OR Q	NOT P	IF P THEN Q
F	T	F	T	T	T
F	F	F	F	T	T

Beginners often find the last two rows of IF-THEN odd. Think of it as a promise: "If it rains, the road is wet." The promise is broken only when it rains and the road is dry. If it is not raining, the promise says nothing, so it is not broken.

Inference means using known statements to reach a new one. The basic valid pattern is **modus ponens**: if "IF P THEN Q" is true and P is true, then Q must be true. This is exactly what a rule-based system does when it fires a rule (Topic 10).

A second valid pattern is **modus tollens**: if "IF P THEN Q" is true and Q is **false**, then P must be false. (If rain always makes the road wet, and the road is dry, it is not raining.)

Two common **invalid** patterns (they look convincing but do not follow):

Pattern	Form	Why it fails
Affirming the consequent	IF P THEN Q; Q is true; so P is true	Something else may have made Q true
Denying the antecedent	IF P THEN Q; P is false; so Q is false	Q may be true for another reason

	Know	Conclude	Valid?
Modus ponens	IF P THEN Q, and P	Q	✔
Modus tollens	IF P THEN Q, and NOT Q	NOT P	✔
Affirming the consequent	IF P THEN Q, and Q	P	✘
Denying the antecedent	IF P THEN Q, and NOT P	NOT Q	✘

Rules with variables

So far each letter stood for one fixed statement. AI rules usually talk about **any** object, using a **variable** such as X: **IF X is a dog THEN X is a mammal**. To use the rule, we **substitute** a specific object for X: with the fact "Rex is a dog", X = Rex, so "Rex is a mammal" follows by modus ponens. This step from one-off statements to statements about "any X" is the idea behind **predicate logic** (also called first-order logic), which Topics 09 and 10 rely on.

Worked Example

Statements: **Rule:** IF it is raining THEN the road is wet. **Fact:** It is raining.

1. Name the parts: P = "It is raining", Q = "The road is wet".
2. We know "IF P THEN Q" is true.
3. We know P is true.
4. By modus ponens, Q is true: **the road is wet**.

Now change the fact to "The road is wet". Can we conclude it is raining? No. A sprinkler or a cleaning truck could also wet the road, so Q being true does not force P to be true.

Now change the fact to "The road is **not** wet". By **modus tollens**, it is **not raining**. And if the fact is "It is **not** raining", we can conclude nothing about the road (denying the antecedent would be invalid).

Activity

1. Rule: "IF a number is divisible by 4 THEN it is even". Fact: 12 is divisible by 4. What follows?
2. Rule: "IF the library is open THEN students can borrow books". Fact: students can borrow books. Can you conclude the library is open?
3. Complete the truth table for **P AND (NOT Q)**.

P	Q	NOT Q	P AND (NOT Q)
T	T		
T	F		
F	T		
F	F		

4. Rule: "IF X is a square THEN X has four sides". (a) Fact: shape K is a square. What follows, and what did X stand for? (b) Fact: shape M does **not** have four sides. What follows, and by which pattern? (c) Fact: shape T is **not** a square. Can you conclude that T does not have four sides?
5. **Find the error.** A student reasons: "IF a student studies THEN they pass. Bat did not study. So Bat failed." Name the mistake and give a situation that shows the conclusion could be wrong.

Answer

1. 12 is even (modus ponens).
2. No. Books might be borrowed through another service, so this is the invalid pattern of affirming the consequent.
3. Rows: T,T → NOT Q = F, result **F**; T,F → T, **T**; F,T → F, **F**; F,F → T, **F**. Only P true and Q false makes the statement true.
4. (a) X = K, so **K has four sides** (modus ponens). (b) **M is not a square** (modus tollens). (c) **No.** A rectangle is not a square but still has four sides; this would be denying the antecedent.
5. **Denying the antecedent.** The rule only says what happens if a student studies; it says nothing about students who do not. Bat might already know the material, or the test might be easy, and still pass.

Common Misunderstandings

- "*OR means one or the other, but not both.*" In logic, P OR Q is also true when **both** are true (row 1 of the truth table).
- "*IF P THEN Q also means IF Q THEN P.*" It does not; reversing a rule is exactly the error of affirming the consequent.

- "*A valid argument always has a true conclusion.*" Validity is about the **form**: if the starting statements are true, the conclusion must be true. If a starting rule is false, a valid argument can still reach a false conclusion.

Quick Self-Check

1. When is IF P THEN Q false? *(Only when P is true and Q is false.)*
2. From IF P THEN Q and NOT Q, what follows? *(NOT P, by modus tollens.)*
3. Is "Close the door!" a proposition? *(No; a command is neither true nor false.)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** chapter "Logic and Reasoning"; truth-table completion, valid/invalid argument questions, rule-with-variable questions.
- **PPT Slides:** connectives table, truth table built row by row, the four-patterns table.
- **Video Tutorial:** not selected as a separate video; modus ponens is explained inside tutorial 3 (expert systems), where it is what "firing a rule" means.

References

1. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Chapters on logical agents and first-order logic.)
2. Rosen, K. H. (2019). *Discrete Mathematics and Its Applications* (8th ed.). McGraw-Hill. (Propositions, truth tables, rules of inference and fallacies.)
3. Foundations of CS & AI 2026, Chapter 4 course slides.

Example sentences are the team's own.

09 Knowledge Representation

Chapter: Chapter 4: Symbolic AI
Status: Drafted (V2.0, 3 Nov 2026; V1.0 24 Oct). V2.0 adds a worked frame example, an "unknown vs false" note, misunderstandings, self-check and reuse notes. Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: Topics 07 and 08 (facts, rules, true/false statements).
Draft note: Prepared with Claude assistance (AI-use records AI-4 and AI-5); to be checked by a team member.

Learning Objectives

1. Explain why a computer needs knowledge written in a structured form.
2. Describe **facts**, **rules**, **semantic networks** and **frames**.
3. Use **inheritance** in a semantic network to answer a question, including an exception.
4. Write a simple **frame** with slots and fill a slot by inheritance.
5. Choose a suitable representation for a piece of knowledge.

Explanation

Knowledge representation means writing what we know about the world in a form a computer can store and use for reasoning. A sentence such as "Dogs are mammals" is easy for people, but a program needs it in a fixed structure.

Four common forms:

- **Fact:** a single true statement, e.g. `Rex is a dog`.
- **Rule:** an IF-THEN statement, e.g. IF X is a dog THEN X is a mammal (used in Topic 10).
- **Semantic network:** a diagram of **nodes** (things or categories) joined by labelled **links**. The most common link is **is-a** (Rex is-a Dog).
- **Frame:** a record for one thing with named **slots**, e.g. Dog: {category: Mammal, legs: 4, sound: bark}.

Inheritance: an item automatically has the properties of the categories above it. If Rex is a Dog, and a Dog is a Mammal, and Mammals have fur, then Rex has fur, even though "Rex has fur" was never stored. This saves space and keeps knowledge consistent.

Exceptions: sometimes a lower node overrides a higher one. Birds can fly, but a penguin is a bird that cannot. The most specific information wins.

Semantic network: is-a links and inherited properties

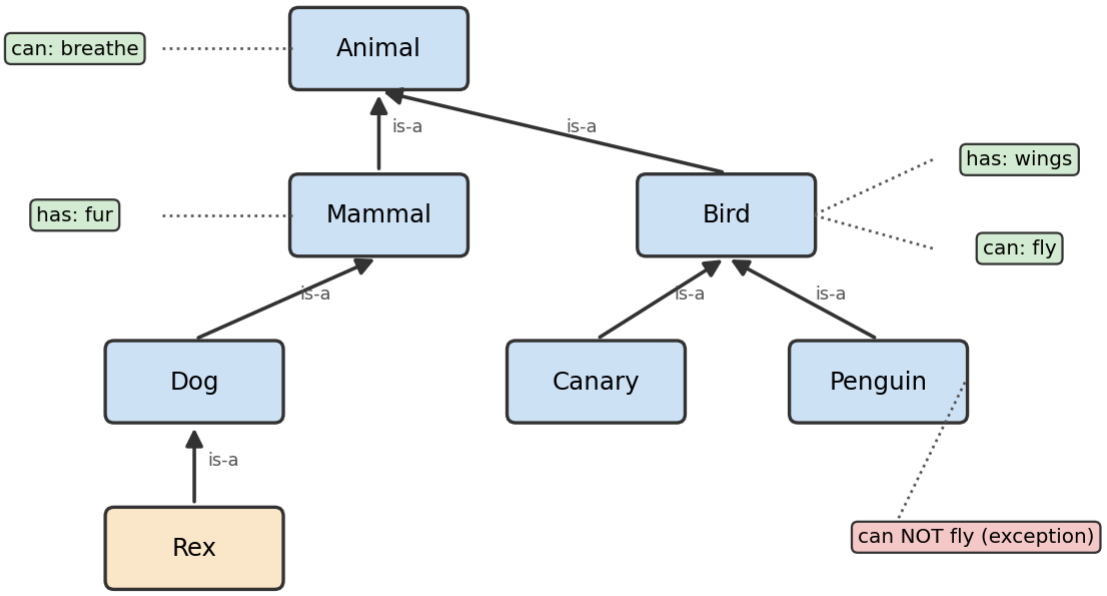


Figure 1. Arrows show is-a links. Green tags are properties stored once at the top and inherited downward. The red tag on Penguin overrides the inherited "can fly".

Comparing the four forms

Form	Best for	Example	Weakness
Fact	Single pieces of information	`Rex is a dog`	Many separate facts are hard to organise
Rule	Knowledge of the form "if … then …" and decisions	IF X is a bird THEN X has wings	Thousands of rules become hard to maintain (Topic 10)
Semantic network	Categories and relationships; inheritance	Rex → Dog → Mammal → Animal	Link meanings must be agreed carefully
Frame	Everything about one kind of object in one place, with default values	Dog frame with slots	Hard to show actions and reasoning steps

Unknown is not the same as false. If the knowledge base does not contain "Rex has wings", there are two possible policies: treat it as **false** (assume that anything not stored is false, called the *closed-world assumption*, as databases in Topic 04 do) or treat it as **unknown**. The choice changes the system's answers, so a designer must decide it on purpose.

Worked Example

Question: **Can Rex breathe?**

1. Start at **Rex**. It has no property "can breathe" stored.
2. Follow the is-a link up: Rex is-a **Dog**. Dog has no "can breathe".
3. Dog is-a **Mammal**. Mammal has "has: fur" but not "can breathe".
4. Mammal is-a **Animal**. Animal has "can: breathe". Found.
5. By inheritance, **Rex can breathe**.

A second question, **Can a penguin fly?** Start at Penguin: "can NOT fly" is stored there. We stop at the first match, so the answer is **no**, even though Bird says "can: fly".

Worked example with frames

Frames are like the records of Topic 06, with one extra power: a slot can be **inherited** from a more general frame.

```
Frame: Bird                      Frame: Penguin
  is-a:  Animal                    is-a:  Bird
  wings: 2                         can_fly: no      (overrides Bird)
  can_fly: yes   (default)         habitat: cold coasts
  covering: feathers

Frame: Pingu
  is-a: Penguin
  age:  3
```

Question: what is Pingu covered with, and can Pingu fly?

1. Look in Pingu's frame for `covering`: not there. Go up the is-a link to **Penguin**: not there. Go up to **Bird**: `covering: feathers`. **Feathers.**
2. Look for `can_fly`: not in Pingu; found in **Penguin**: `no`. Stop at the first (most specific) value. **Pingu cannot fly**, even though Bird's default is "yes".

Activity

Use Figure 1.

1. Does Rex have fur? Write the path you followed.
2. Does a canary have wings? Does a canary fly?
3. Does Rex have wings?
4. Design question: add **Cat** to the network and give it one property of its own. Which links and properties must you add? What can a cat now inherit without storing it again?
5. **Frames.** Using the frames in the worked example: (a) how many wings does Pingu have? Which frame supplied the answer? (b) Write a frame for **Ostrich** that inherits from Bird and overrides one slot.
6. **Choose a representation.** Which form fits best, and why? (a) "If a book is more than 14 days overdue, the student pays a fine." (b) Everything a phone shop needs to store about one phone model: brand, screen size, price, colour. (c) "A sparrow is a bird; a bird is an animal."

Answer

1. Yes. Rex → Dog → Mammal; Mammal stores "has: fur".
2. Canary → Bird; Bird stores "has: wings" and "can: fly". So yes to both.

3. No information. Rex → Dog → Mammal → Animal never reaches "has: wings", so the system cannot say Rex has wings. (A real system would treat this as "not known" or "no".)
4. Add the node **Cat** and one link **Cat is-a Mammal**. Add one own property, for example "sound: meow". Cat now inherits "has: fur" from Mammal and "can: breathe" from Animal without storing them again.
5. (a) **2 wings**, inherited from the **Bird** frame (Pingu → Penguin → Bird). (b) Example: `Frame: Ostrich · is-a: Bird · can_fly: no · top_speed: 70 km/h`. Any frame with an is-a link to Bird and one slot that overrides a Bird default (such as can_fly) is correct.
6. (a) **Rule**: it is a condition leading to an action. (b) **Frame** (or a record): many named properties of one kind of object. (c) **Semantic network** with is-a links, so properties can be inherited.

Common Misunderstandings

- "*Inheritance copies the properties down into every node.*" Nothing is copied; the property is stored once and **found** by following the is-a links when needed.
- "*If a fact is not stored, it is false.*" Only under the closed-world assumption. Many AI systems treat missing knowledge as **unknown**.
- "*The arrow in a semantic network can mean anything.*" Each link label (is-a, has-part, eats) has one agreed meaning; mixing meanings leads to wrong inferences.

Quick Self-Check

1. What is the most common link in a semantic network? *(is-a.)*
2. When a lower node and a higher node give different values, which wins? *(The lower, more specific node.)*
3. What are the named fields of a frame called? *(Slots.)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** chapter "Knowledge Representation"; inheritance-path questions, frame-slot questions, choose-a-representation questions.
- **PPT Slides:** four-forms comparison table, Figure 1 with the inheritance path highlighted, Pingu frames.
- **Video Tutorial:** not selected as a separate video in Stage 2; Figure 1 is a candidate for a short animated clip in Stage 3.

References

1. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Knowledge representation chapters.)
2. Minsky, M. (1975). A framework for representing knowledge. In P. H. Winston (Ed.), *The Psychology of Computer Vision* (pp. 211–277). McGraw-Hill. (Frames.)
3. Foundations of CS & AI 2026, Chapter 4 course slides.

The animal network is the team's own example.

10 Rule-Based and Expert Systems

Chapter: Chapter 4: Symbolic AI
Status: Drafted (V2.0, 3 Nov 2026; pilot topic, V1.0 24 Oct). V2.0 adds conflict resolution, misunderstandings and links to Topics 08, 11 and 13. Waiting for second-member review (see `tasks.md`, T12–T13).
Owner: Rawa Rashid (content lead). **Reviewer:** see `tasks.md` (T12)
Prerequisites: Topics 07–09: what Symbolic AI is, what a fact and a rule are, and IF-THEN logic.
Draft note: Prepared with Claude assistance (AI-use records AI-4 and AI-5). Examples were run and checked with a short program. Humans on the team must still review and be able to explain every part.

Learning Objectives

After this topic, a beginner can:

1. Define **rule**, **fact**, **knowledge base**, **working memory**, **inference engine** and **expert system**.
2. Use **forward chaining** to work out, step by step, what a set of rules and facts lets us conclude.
3. Use **backward chaining** to test whether a goal can be proved.
4. Explain two strengths and two limits of rule-based systems.

Explanation

What is an expert system?

An **expert system** is a computer program that stores the knowledge of a human expert about one narrow subject and uses it to give advice, such as diagnosing a fault. Instead of learning from data, it follows rules that people wrote down. This is why it counts as **Symbolic AI**: the knowledge is written in readable symbols (words and rules), not hidden inside numbers.

Two historic examples:

- **MYCIN** (Stanford, 1970s) suggested antibiotic treatments for blood infections using several hundred IF-THEN rules. It was a research system and was not used in routine hospital care.
- **XCON** (also called R1, used by Digital Equipment Corporation in the 1980s) checked customer orders for computers and decided how to configure the parts.

Key terms

Term	Meaning	Laptop example
Fact	A statement that is known to be true right now	"The power light is off"

Term	Meaning	Laptop example
Rule	An IF-THEN statement: if all the conditions are true, the conclusion becomes true	IF power light is off AND charger is plugged in THEN no power reaches the laptop
Knowledge base	The collection of all rules, written by experts	Rules R1–R7 below
Working memory	The facts known at the moment, including new conclusions the system adds	F1–F4, then F5, F6 and so on
Inference engine	The part that decides which rule to apply next and adds the conclusion	The "reasoning" loop
Firing a rule	Using a rule whose conditions are all true to add its conclusion	R1 fires and adds F5

How the parts fit together

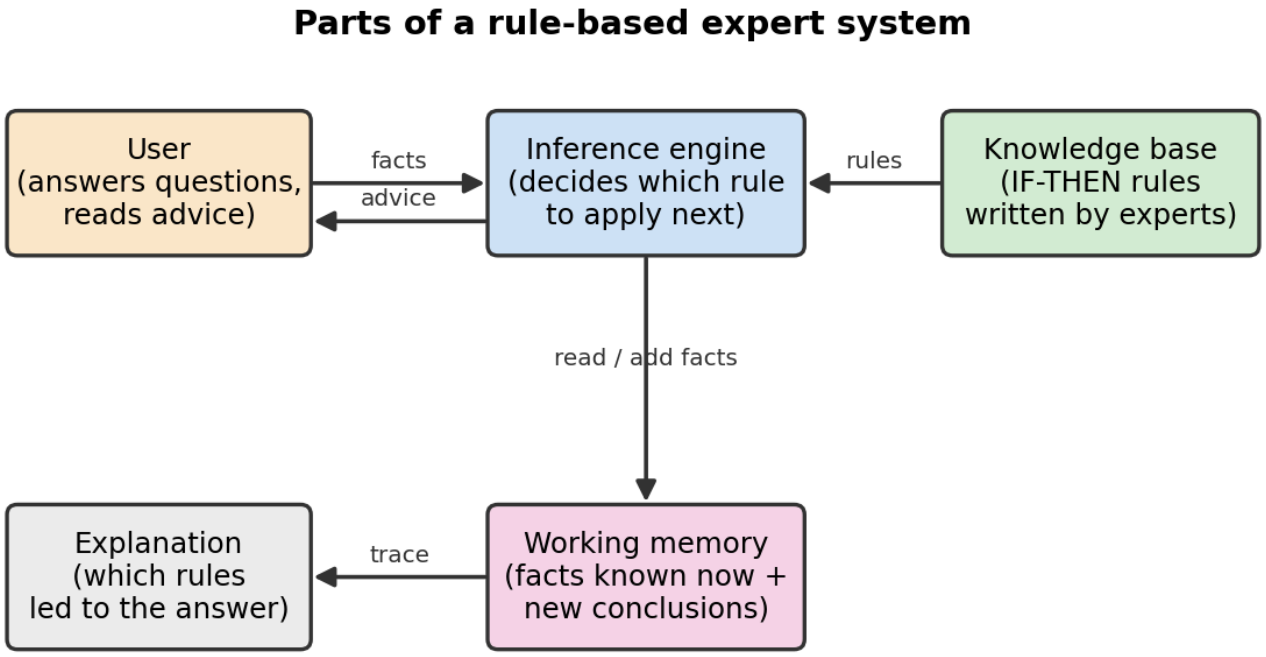


Figure 1. The user gives facts. The inference engine reads the rules from the knowledge base and the facts from working memory, adds new conclusions, and gives advice. Because every step is a rule, the system can also explain which rules led to its answer.

Two ways to reason

Forward chaining (data-driven) starts from the facts and works toward a conclusion.

1. Look for a rule whose conditions are all in working memory and whose conclusion is not yet known.
2. Fire it: add its conclusion to working memory.
3. Repeat from step 1.
4. Stop when a goal is reached or no rule can fire.

Backward chaining (goal-driven) starts from a goal and works backward to see whether the facts support it.

1. Pick the goal you want to prove.
2. If it is already a fact, it is proved.
3. Otherwise find a rule that concludes it. Its conditions become new sub-goals. Prove each one the same way.
4. If a sub-goal is not a fact and no rule concludes it, that route fails.

	Forward chaining	Backward chaining
Starts from	Known facts	A goal
Direction	Facts → conclusions	Goal → facts
Good when	You have data and want to know "what follows?"	You have one question and want to know "can I prove this?"
Typical use	Monitoring, configuration	Diagnosis, question answering

Both methods use the same rules and reach the same proved conclusions. They differ in the direction of search.

Connections. Firing a rule is exactly **modus ponens** from Topic 08: IF conditions THEN conclusion, the conditions are true, so the conclusion is true. Backward chaining is a kind of **search** (Topic 11): each rule that could prove a goal is a branch to explore.

When more than one rule can fire

Sometimes several rules have all their conditions true in the same cycle. The inference engine needs a **conflict-resolution strategy** to choose one. Common strategies:

- **Rule order:** fire the rule that comes first in the knowledge base (used in this topic's examples).
- **Specificity:** prefer the rule with more conditions, because it fits the situation more exactly.
- **Recency:** prefer the rule that uses the newest facts.

In the laptop example only one new rule is ready in each cycle, so the choice never matters; in large systems it can change the order of conclusions.

Worked Example: A laptop will not start

Knowledge base (rules)

Rule	IF	THEN
R1	power light is off AND charger is plugged in	no power reaches the laptop
R2	no power reaches the laptop	suspect charger or battery
R3	suspect charger or battery AND another charger works	charger is faulty
R4	suspect charger or battery AND another charger does not work	battery is faulty

Rule	IF	THEN
R5	power light is on AND screen is black	screen or display problem
R6	charger is faulty	advice: replace the charger
R7	battery is faulty	advice: replace the battery

Facts the user gives (working memory at the start)

- F1: screen is black
- F2: power light is off
- F3: charger is plugged in
- F4: another charger works

Forward chaining, step by step

Cycle	Rules whose conditions are all true	Rule fired	New fact added
1	R1 (F2 and F3). R5 fails because the power light is not on	R1	F5: no power reaches the laptop
2	R2 (F5)	R2	F6: suspect charger or battery
3	R3 (F6 and F4). R4 fails because "another charger does not work" is not a fact	R3	F7: charger is faulty
4	R6 (F7)	R6	F8: advice: replace the charger
5	None (R7 needs "battery is faulty", which is not known)	Stop	—

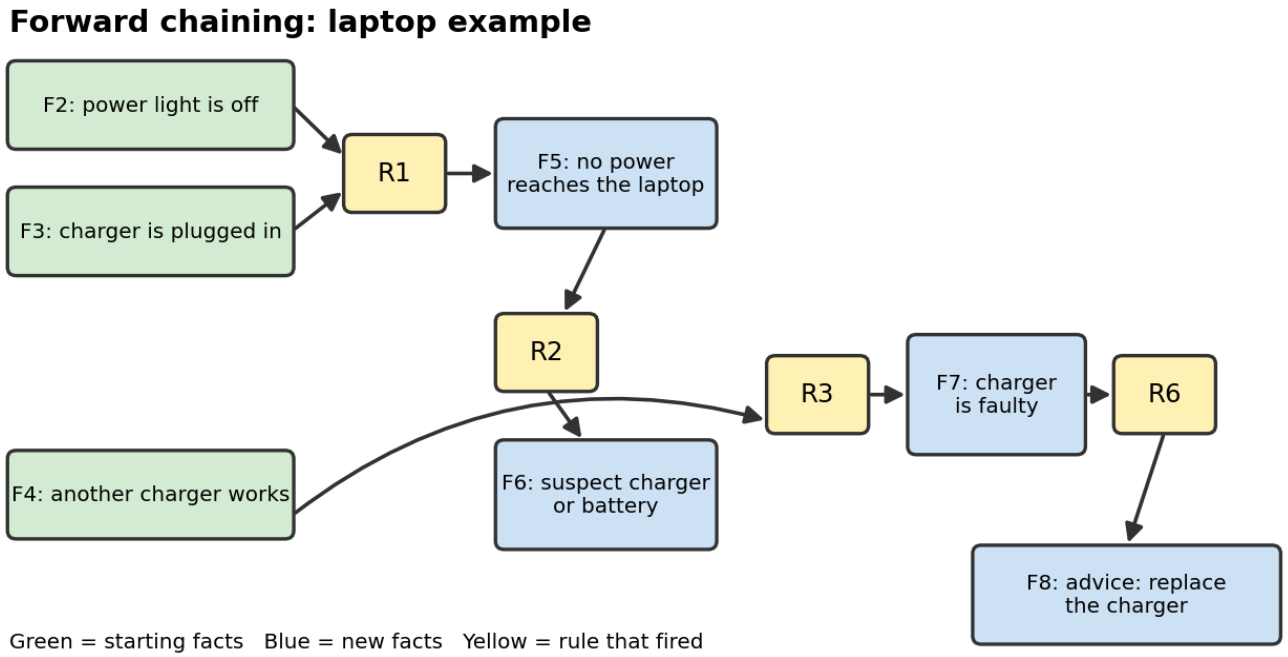


Figure 2. How each rule turns facts into a new fact. F1 ("screen is black") is never used because only R5 needs it and R5 does not fire.

Backward chaining, step by step

Goal: advice: replace the charger.

1. The goal is not a fact. R6 concludes it, so we need **charger is faulty**.
2. R3 concludes "charger is faulty". It needs **suspect charger or battery** and **another charger works**.
3. "Another charger works" is a fact (F4). ✔
4. R2 concludes "suspect charger or battery". It needs **no power reaches the laptop**.
5. R1 concludes that. It needs **power light is off** (F2 ✔) and **charger is plugged in** (F3 ✔).
6. Every sub-goal is proved, so the goal is proved.

A goal that fails: advice: replace the battery. R7 needs "battery is faulty". R4 would conclude that, but it needs "another charger does not work". That is not a fact and no rule concludes it. The route fails, so the system cannot advise replacing the battery.

Activity

Use the same rules R1–R7.

1. **Forward chaining.** The user gives only: F1 screen is black, F2 power light is **on**. Which rules fire and what does the system conclude? Does it give any advice?
2. **Improve the knowledge base.** Write one new rule, R8, so that the system in question 1 can give advice. Then run it again.
3. **Backward chaining.** Facts: power light is off, charger is plugged in, another charger does **not** work. Use backward chaining to test the goal "advice: replace the battery". Write each sub-goal.
4. **Apply the idea.** Choose a small everyday decision (for example "Should I bring an umbrella?" or "Which study room should I book?"). Write three facts and three IF-THEN rules that lead to a recommendation. Then write the forward-chaining steps for your example.

Answer

1. Only R5 fires (power light is on AND screen is black). New fact: **screen or display problem**. No rule uses that fact, so the system stops with **no advice**. This shows a limit: the knowledge base has no rule for this case.
2. Example: **R8: IF screen or display problem THEN advice: connect an external monitor to test the screen.** Now R5 fires first, then R8 fires, and the system gives advice. Any rule that links "screen or display problem" to a sensible action is acceptable.
3. Goal: advice: replace the battery → needs R7: **battery is faulty** → needs R4: **suspect charger or battery** and **another charger does not work**. "Another charger does not work" is a fact ✔. "Suspect charger or battery" needs R2 → **no power reaches the laptop** → R1 → power light is off ✔ and charger is plugged in ✔. All sub-goals are proved, so the system advises replacing the battery.
4. Open answer. A good answer: (a) facts are things that can be checked as true or false; (b) each rule has IF conditions and one THEN conclusion; (c) the conclusion of one rule can be a condition of another rule; (d) the forward-chaining steps show each rule fired in

order and the new fact it adds. Example: facts "sky is cloudy", "forecast says rain"; rule "IF sky is cloudy AND forecast says rain THEN rain is likely"; rule "IF rain is likely THEN advice: bring an umbrella".

Strengths and Limits

Strengths

- *Transparent:* the system can show the exact rules behind its answer.
- *Consistent:* the same facts always give the same conclusion.

Limits

- *Knowledge bottleneck:* experts must write all the rules by hand, which is slow, and the experts may not be able to put all their knowledge into rules.
- *Brittle:* outside its narrow subject, or when a rule is missing (Activity question 1), the system fails instead of guessing.
- *Hard to maintain:* with thousands of rules, adding one rule can conflict with others.
- *No learning:* it only knows what people wrote. This is different from machine learning, which finds patterns in data.

Common Misunderstandings

- "*The expert system understands laptops.*" It only matches symbols in rules to symbols in working memory. "Charger is faulty" means nothing to it beyond the rules that mention it.
- "*If the system gives no advice, the facts must be wrong.*" Often the **knowledge base** has a gap (Activity question 1).
- "*Forward chaining uses every fact.*" It only uses facts that appear in rule conditions; F1 was never used.

Quick Self-Check

1. Which part of an expert system decides which rule to use next? *(Inference engine)*
2. You have a single question to prove, such as "is the charger faulty?". Which method is the natural choice? *(Backward chaining)*
3. True or false: forward and backward chaining can reach different proved conclusions from the same rules and facts. *(False; they differ in direction, not in what can be proved.)*

How This Topic Will Be Reused

- **Study Guide:** becomes a chapter with the same sections: terms table, two ways to reason, worked example, activity, answers.
- **PPT Slides:** Figure 1 (parts of the system), Figure 2 (rule chain) and the forward-chaining table, one idea per slide.
- **Video Tutorial:** "How a rule-based expert system reaches a conclusion": narrate the forward-chaining table cycle by cycle, then the backward-chaining steps.

References

1. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Sections on logical agents and inference; forward and backward chaining.)
2. Buchanan, B. G., & Shortliffe, E. H. (Eds.). (1984). *Rule-Based Expert Systems: The MYCIN Experiments of the Stanford Heuristic Programming Project*. Addison-Wesley. (MYCIN.)
3. McDermott, J. (1982). R1: A rule-based configurer of computer systems. *Artificial Intelligence, 19*(1), 39–88. (XCON/R1.)
4. Jackson, P. (1998). *Introduction to Expert Systems* (3rd ed.). Addison-Wesley. (Expert system parts and chaining.)
5. Foundations of CS & AI 2026, Chapter 4 course slides. (Course coverage of rule-based systems.)

The laptop rules, facts and diagrams are the team's own examples, not taken from the sources above.

11 Planning and Search

Chapter: Chapter 4: Symbolic AI
Status: Drafted (V2.0, 3 Nov 2026). Both searches on both maps were run with a short program and match the hand traces. Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: Topic 05 (step-by-step algorithms), Topic 06 (stack, queue, tree, graph), Topic 09 (knowledge stored as nodes and links).
Draft note: Prepared with Claude assistance (AI-use record AI-5). Every search order and path below was produced by code and compared with the hand trace.

Learning Objectives

After this topic, a beginner can:

1. Describe a problem as **states**, **actions**, a **start state** and a **goal**.
2. Explain what a **state space** and a **search tree** are.
3. Carry out **breadth-first search (BFS)** and **depth-first search (DFS)** by hand on a small graph.
4. Compare BFS and DFS: which finds the shortest route, and which uses less memory.
5. Explain what a **plan** is in AI and give an everyday planning example.

Explanation

Problems as states and actions

Many AI problems are "get from here to there": find a route, solve a puzzle, plan a robot's moves. Symbolic AI describes them in a fixed way:

Term	Meaning	Route example	Puzzle example (water jugs)
State	A snapshot of the situation	"I am in town C"	"4-litre jug has 2 L, 3-litre jug has 0 L"
Start state	Where we begin	Town A	Both jugs empty
Action	A move that changes one state into another	Drive along a road to a neighbouring town	Fill a jug, empty a jug, pour one into the other
Goal	The state (or test) we want to reach	Being in town F	Exactly 2 L in the 4-litre jug
Path / solution	A sequence of actions from start to goal	A → B → D → F	Fill, pour, …

All the states and the actions linking them form a **state space**, which is a **graph** (Topic 06): states are nodes, actions are edges. **Search** means exploring this graph in a systematic way until a goal is found.

Search tree and frontier

As the search explores, it builds a **search tree**: the start state is the root, and each node's children are the states reachable by one action. The **frontier** is the list of states found but **not yet explored**. The only difference between BFS and DFS is **which frontier state is explored next**:

	Breadth-first search (BFS)	Depth-first search (DFS)
Explores next	The **oldest** state in the frontier	The **newest** state in the frontier
Frontier structure	**Queue** (FIFO)	**Stack** (LIFO)
Shape of exploration	Level by level: all places 1 step away, then 2 steps, …	Goes as deep as possible along one branch, then backs up
Finds the route with fewest steps?	**Yes** (always, if one exists)	**Not guaranteed**
Memory needed	Can be large: must remember a whole level	Usually small: remembers one branch

The general procedure (with a "visited" list so no state is explored twice):

1. Put the start state in the frontier.
2. Take one state out of the frontier (BFS: the oldest; DFS: the newest).
3. If it is the goal, stop and trace the path back.
4. Otherwise add its **unvisited** neighbours to the frontier, remembering which state you came from.
5. If the frontier is empty, there is no solution. Otherwise go to step 2.

Tie-break rule used in this module: when several neighbours could be added, take them in **alphabetical order**. Without a fixed rule, different people get different (but still correct) orders.

Planning

A **plan** is a sequence of actions that takes the world from the start state to a goal. **Planning** is search where the states describe the world using symbols, and each action has **preconditions** (what must be true before) and **effects** (what becomes true after). For example:

Action	Preconditions	Effects
Boil water	have kettle, have water	water is hot
Make tea	water is hot, have tea bag, have cup	tea is ready

Goal: *tea is ready*. A planner searches for actions whose effects achieve the goal and whose preconditions can be met: **Boil water → Make tea**. This is the same backward idea as backward chaining in Topic 10. The STRIPS planner (Fikes and Nilsson, 1971) used exactly this description, and planners like it are used in logistics, robotics and space missions.

Worked Example: From A to F on a small map

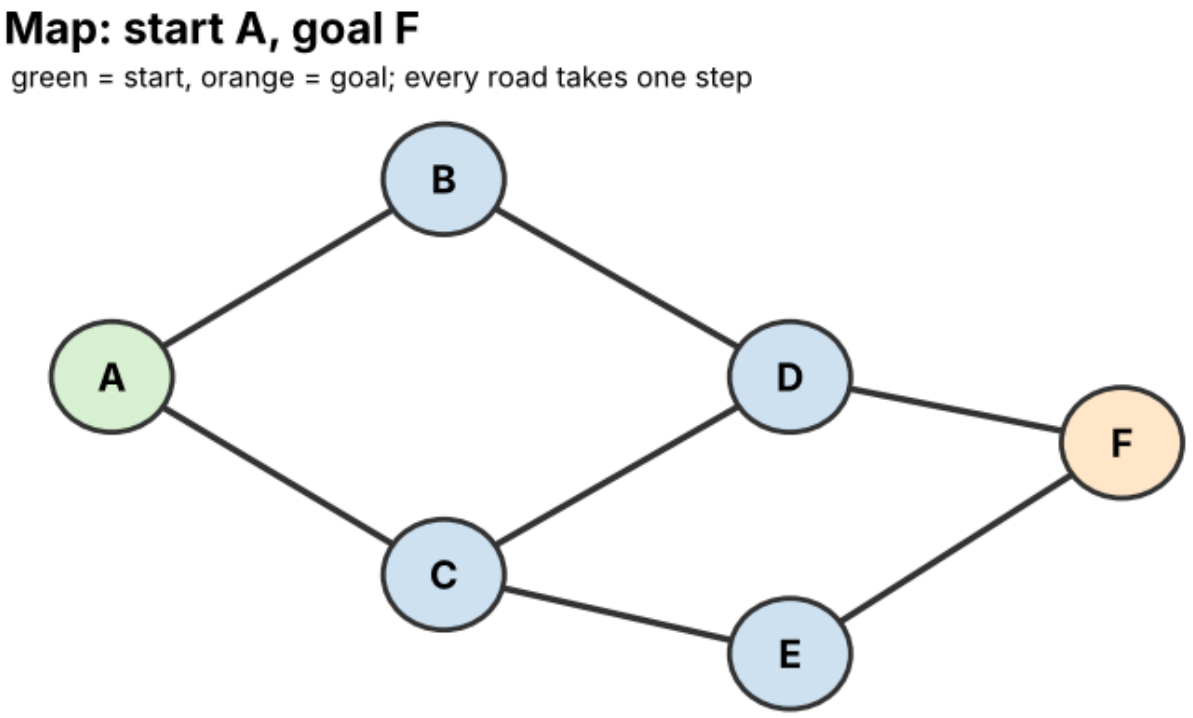


Figure 1. Roads (edges) between six towns (nodes). Start: A. Goal: F.

Neighbours (alphabetical): A: B, C · B: A, D · C: A, D, E · D: B, C, F · E: C, F · F: D, E.

Breadth-first search (frontier is a queue)

Step	Take out	New unvisited neighbours added (came from)	Frontier queue afterwards (front → back)
1	A	B (A), C (A)	B, C
2	B	D (B)	C, D
3	C	E (C) — D already found	D, E
4	D	F (D)	E, F
5	E	none (C and F already found)	F
6	F	**Goal!**	–

Visiting order: **A, B, C, D, E, F**. Trace back from F: F came from D, D from B, B from A. **Path: A → B → D → F (3 roads).** BFS guarantees no route with fewer roads exists.

Depth-first search (always go deeper first)

Rule: from the current town, go to the first unvisited neighbour (alphabetical). If there is none, go back one step.

Step	At	Unvisited neighbours	Move to
1	A	B, C	B
2	B	D	D

Step	At	Unvisited neighbours	Move to
3	D	C, F	C
4	C	E	E
5	E	F	F → **Goal!**

Visiting order: **A, B, D, C, E, F**. **Path: A → B → D → C → E → F (5 roads).** DFS found *a* route quickly with little memory, but not the shortest one: at D it went to C (alphabetically first) instead of F, which was right next to it.

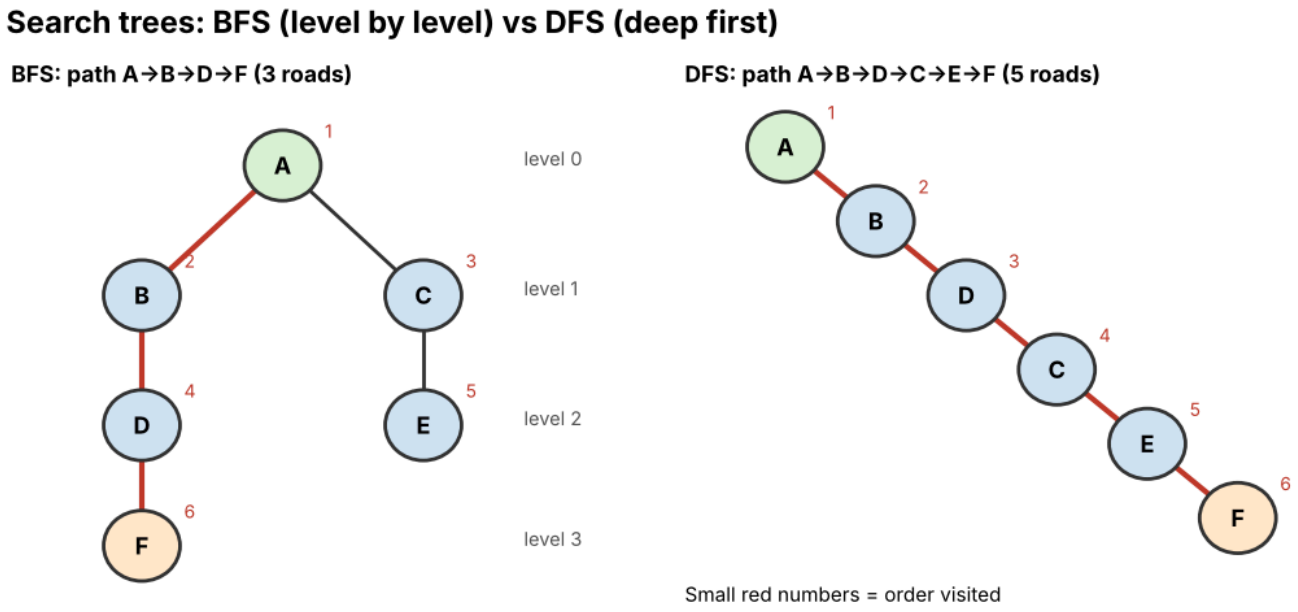


Figure 2. The search trees. BFS explores level by level; DFS dives down one branch.

Activity

Use this new map. Start: **S**. Goal: **G**. Roads: S–A, S–B, A–C, A–D, B–G, C–D, D–G.

Neighbours (alphabetical): S: A, B · A: C, D, S · B: G, S · C: A, D · D: A, C, G · G: B, D.

1. **BFS.** Run breadth-first search. Write a table like the worked example (take out, added, queue). Give the visiting order and the path found.
2. **DFS.** Run depth-first search with the alphabetical rule. Give the visiting order and the path found.
3. **Compare.** Which method found the shorter route? Explain why, using the words *queue*, *stack* and *level*.
4. **State the problem.** Describe the problem "a student must get from the dormitory to the exam hall by bus or on foot" in terms of states, actions, start state and goal.
5. **Plan.** Using preconditions and effects, write two actions that achieve the goal *essay submitted*, starting from *have notes*. Then write the plan.

Answer

1.

Step	Take out	Added (came from)	Queue afterwards
1	S	A (S), B (S)	A, B
2	A	C (A), D (A)	B, C, D
3	B	G (B)	C, D, G
4	C	none	D, G
5	D	none (G already found)	G
6	G	**Goal**	–

Visiting order: **S, A, B, C, D, G**. Path (trace back G ← B ← S): **S → B → G (2 roads).** 2. S → A (first neighbour) → C (A's first unvisited) → D (C's only unvisited) → G (D's unvisited). Visiting order: **S, A, C, D, G**. Path: **S → A → C → D → G (4 roads).** 3. **BFS** found the shorter route (2 roads vs 4). BFS uses a **queue**, so it finishes every place one road from S (A and B) before any place two roads away; G is at **level** 2, so it is found by the shortest route. DFS uses a **stack** (always the newest place), so it went deep along A → C → D before ever trying B. 4. States: the student's location (dormitory, bus stop, road junctions, exam hall) — and possibly the time. Actions: walk to a neighbouring place; take a bus between two stops. Start state: at the dormitory. Goal: at the exam hall (perhaps "before 9:00"). 5. Example: **Write essay** — preconditions: have notes; effects: essay written. **Submit essay** — preconditions: essay written, have internet; effects: essay submitted. Plan: **Write essay → Submit essay** (assuming "have internet" is true at the start). Any pair where the first action's effect provides the second action's precondition is correct.

Common Misunderstandings

- "*DFS is wrong because it found a long route.*" DFS is a correct search method; it finds *a* solution, just not always the shortest. Its strength is low memory.
- "*Search means looking through a list, like Topic 05.*" Here search explores a **graph of states**, where each step creates new possibilities.
- "*The computer can see the whole map at once and pick the best route.*" A search procedure only knows the states it has explored so far; that is why the order of exploration matters.

Quick Self-Check

1. Which data structure holds the frontier in BFS? In DFS? *(Queue; stack.)*
2. Which method guarantees the route with the fewest steps? *(BFS.)*
3. In planning, what must be true before an action can be used? *(Its preconditions.)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** chapter "Planning and Search"; BFS/DFS trace questions on new maps; states/actions/goal questions.
- **PPT Slides:** state-space terms table, Figure 1, BFS queue table step by step, DFS table, Figure 2.
- **Video Tutorial:** "Search through a state space" (planned tutorial 4): BFS and DFS on the A–F map, side by side.

References

1. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Chapter 3: problem formulation, BFS and DFS; Chapter 11: classical planning.)
2. Fikes, R. E., & Nilsson, N. J. (1971). STRIPS: A new approach to the application of theorem proving to problem solving. *Artificial Intelligence, 2*(3–4), 189–208.
3. Foundations of CS & AI 2026, Chapter 3 course slides (sample learning material "BFS") and Chapter 4 course slides.

Both maps and the planning examples are the team's own.

12 Symbolic NLP

Chapter: Chapter 4: Symbolic AI
Status: Drafted (V2.0, 3 Nov 2026). The grammar and the chatbot rules were tested with a short program. Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: Topic 06 (trees), Topic 08 (rules), Topic 09 (symbols).
Depth: Brief introduction (B) as agreed in the project plan.
Draft note: Prepared with Claude assistance (AI-use record AI-5). The parses and chatbot replies below were produced by code and compared with the hand traces.

Learning Objectives

After this topic, a beginner can:

1. Explain what **natural language processing (NLP)** is and why language is hard for computers.
2. Describe the steps **tokenisation**, **tagging** and **parsing**.
3. Use a small set of **grammar rules** to build a **parse tree** for a short sentence, and decide whether a sentence is grammatical under those rules.
4. Explain how a **rule-based chatbot** chooses a reply, and find a question it cannot handle.

Explanation

What is NLP?

A **natural language** is a language people speak, such as English, Mongolian or Chinese, as opposed to a programming language (Topic 02). **Natural language processing (NLP)** is the part of AI that lets computers work with human language: translation, search engines, spell-checkers, voice assistants, chatbots.

Language is hard for computers because it is **ambiguous**: one sentence can have several meanings. "I saw the man with the telescope" — who has the telescope? People use context and common sense; a program must be told how to decide.

Symbolic NLP handles language with hand-written **rules**: dictionaries of words and grammar rules written by linguists. (Modern systems such as ChatGPT are mostly data-driven, learning from huge amounts of text; see Topic 07.)

Three basic steps

Step	What it does	"The cat sat."
Tokenisation	Split the text into words and punctuation (**tokens**)	The · cat · sat · .
Tagging	Label each token with its **part of speech** using a **lexicon** (a word list)	The = Det (determiner), cat = N (noun), sat = V (verb)

Step	What it does	"The cat sat."
Parsing	Use **grammar rules** to group the words into phrases and build a **parse tree**	see below

Grammar rules

A **grammar** is a set of rules saying how smaller parts combine into bigger ones. The arrow → means "is made of".

Rule	Meaning
G1: S → NP VP	A **sentence** is a noun phrase followed by a verb phrase
G2: NP → Det N	A **noun phrase** is a determiner followed by a noun ("the cat")
G3: VP → V	A **verb phrase** can be just a verb ("sat")

Lexicon: Det: *the, a* · N: *cat, dog, mat* · V: *sat, slept*

Parsing means finding a way to apply the rules so that the whole sentence becomes one **S**. If no way exists, the sentence is **not grammatical** under this grammar. The result is drawn as a **parse tree** (a tree from Topic 06): S at the root, words at the leaves.

Worked Example: Parsing "The cat sat"

1. **Tokenise:** `The cat sat.`
2. **Tag with the lexicon:** The → Det, cat → N, sat → V. Sequence: **Det N V**.
3. **Apply G2 (NP → Det N):** "The cat" becomes an NP. Sequence: **NP V**.
4. **Apply G3 (VP → V):** "sat" becomes a VP. Sequence: **NP VP**.
5. **Apply G1 (S → NP VP):** the whole thing becomes **S**. Parsing succeeds.

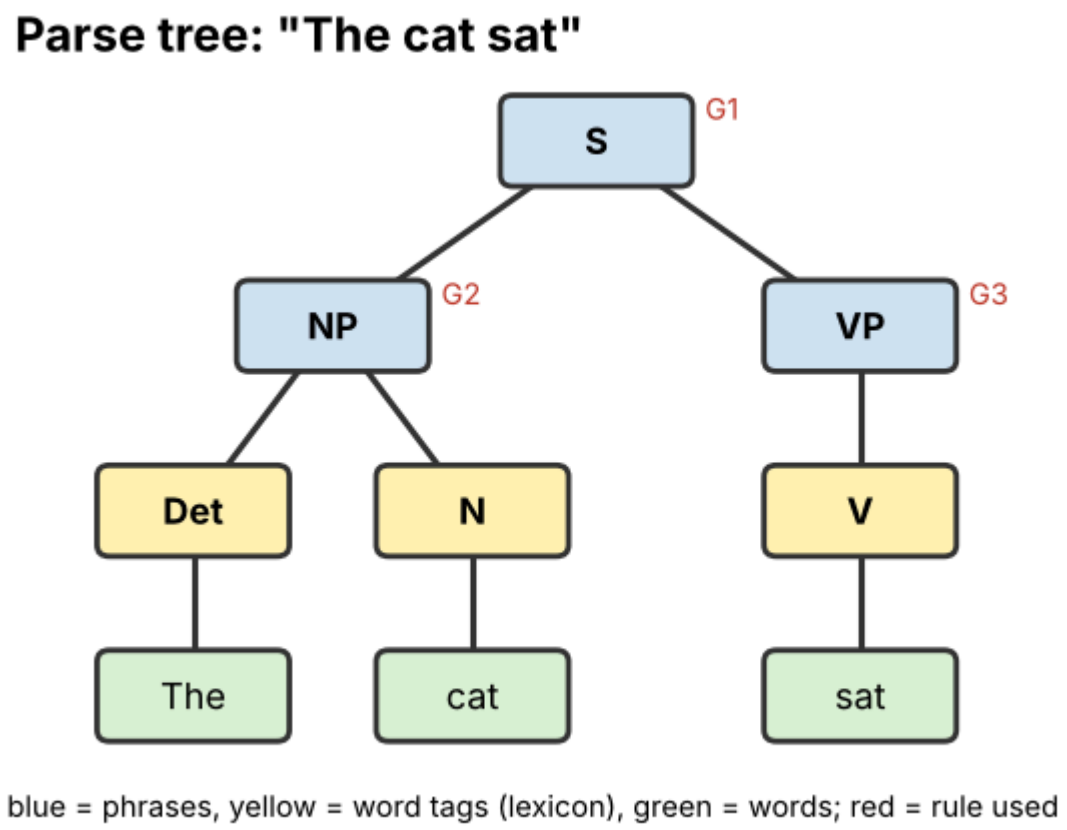


Figure 1. The parse tree. Reading the leaves left to right gives the sentence; each branching point is one grammar rule.

A failing example: "Cat the sat" → tags **N Det V**. G2 needs Det *before* N, so no NP can be formed, and G1 can never apply. The parser rejects the sentence. Notice that the program does not "understand" anything; it only checks that the symbols fit the rules, exactly like the rule matching in Topic 10.

Rule-based chatbots

A simple **chatbot** uses **pattern → response** rules: if the user's message contains a keyword (pattern), give the matching reply. The most famous early example is **ELIZA** (Joseph Weizenbaum, MIT, 1966), which imitated a therapist by turning the user's words back into questions ("I feel sad" → "Why do you feel sad?"). Many people felt ELIZA understood them, although it only matched patterns.

Example: a canteen help bot

Rule	IF the message contains	THEN reply
C1	"open" or "hours"	"The canteen is open 7:00–20:00."
C2	"menu" or "food"	"Today's menu is on the screen by the door."
C3	"pay" or "card"	"You can pay with your student card or WeChat Pay."
Default	(nothing matched)	"Sorry, I don't understand. Please ask staff."

The bot checks the rules **in order** and uses the **first** one that matches.

- "What are your opening hours?" → contains "open" → **C1**.
- "Can I pay by card?" → contains "pay" → **C3**.
- "Is the food open to visitors?" → contains "open" → **C1** fires first, giving the opening hours, which is the wrong answer. The bot matched words, not meaning.

Strengths and limits of symbolic NLP

Strengths	Limits
Predictable and explainable: you can see which rule fired	Real language has far too many patterns to write by hand
Works with little data; good for narrow, fixed tasks (form filling, command words, grammar checkers)	Brittle: spelling mistakes, slang and new phrasing break it
Easy to correct one specific error by editing a rule	Keywords ignore meaning, negation and context ("I do **not** want to pay")

Activity

1. **Parse.** Using G1–G3 and the lexicon, parse **"A dog slept"**. Write the tag sequence and each rule applied. Is it grammatical?
2. **Predict.** Is **"The mat sat"** accepted by the grammar? Is it sensible? What does this tell you about grammar rules?
3. **Extend the grammar.** The sentence **"The cat sat on the mat"** is rejected by G1–G3. Add the word *on* to the lexicon as P (preposition) and write two new rules so that it can be parsed. Then list the rules used.
4. **Write a chatbot.** Write three keyword rules (plus a default) for a **library help chatbot**. Then write one sensible question that your bot will answer **wrongly** or not at all, and explain why.
5. **Find the error.** Using the canteen bot, what reply does "I don't want to pay by card, can I use cash?" get? Why is that a problem?

Answer

1. Tags: A → Det, dog → N, slept → V: **Det N V**. G2: "A dog" → NP. G3: "slept" → VP. G1: NP VP → S. **Grammatical.**
2. **Yes**, it is accepted: The → Det, mat → N, sat → V, and G2, G3, G1 apply. But it is **not sensible** (a mat cannot sit). Grammar rules check **structure** (syntax), not **meaning** (semantics). Handling meaning needs extra knowledge, such as "only animals and people can sit" — the kind of knowledge from Topic 09.
3. Add **G4: VP → V PP** (a verb phrase can be a verb followed by a prepositional phrase) and **G5: PP → P NP** (a prepositional phrase is a preposition followed by a noun phrase). Parse: The cat → NP (G2); the mat → NP (G2); on + NP → PP (G5); sat + PP → VP (G4); NP + VP → S (G1). Rules used: **G2, G2, G5, G4, G1**.
4. Example: L1: "borrow" or "loan" → "You can borrow up to 5 books for 3 weeks." L2: "open" or "hours" → "The library is open 8:00–22:00." L3: "print" → "Printers are on the 2nd floor." Default → "Please ask at the front desk." Question it fails: "How long can I **keep** a book?" has no keyword, so it gets the default reply even though L1 holds the

answer (a different word for the same meaning). Or "Is the printer open on Sunday?" fires L2 (opening hours of the library), not printing information. Any example with a clear explanation of the keyword problem is correct.

5. "pay" is found first → **C3**: "You can pay with your student card or WeChat Pay." It ignores the negation "don't" and never answers the real question about cash. Keyword matching does not understand meaning.

Common Misunderstandings

- "*If the parser accepts a sentence, the computer understands it.*" Parsing only checks structure. Understanding needs meaning and world knowledge.
- "*Chatbots like ELIZA think.*" ELIZA only matched patterns and filled in templates; the feeling of being understood came from the human user.
- "*Symbolic NLP is no longer used.*" Rule-based methods are still used where predictability matters, such as grammar checkers, form parsers and voice commands with a fixed vocabulary, often alongside data-driven models.

Quick Self-Check

1. What is the step that splits text into words called? *(Tokenisation.)*
2. In the rule S → NP VP, what does NP stand for? *(Noun phrase.)*
3. Why did "Is the food open to visitors?" get the wrong reply? *(The bot matched the keyword "open", not the meaning.)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** short chapter; parsing questions and "find the chatbot failure" questions.
- **PPT Slides:** three-steps table, grammar rules, Figure 1 parse tree built step by step, canteen bot table.
- **Video Tutorial:** not selected for video (brief topic).

References

1. Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing* (3rd ed. draft). https://web.stanford.edu/~jurafsky/slp3/ (Tokenisation, parts of speech, context-free grammars, parsing, chatbots and ELIZA.)
2. Weizenbaum, J. (1966). ELIZA—a computer program for the study of natural language communication between man and machine. *Communications of the ACM, 9*(1), 36–45.
3. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Chapter on natural language processing: grammars and parsing.)
4. Foundations of CS & AI 2026, Chapter 4 course slides.

The grammar, sentences, canteen bot and library bot are the team's own examples.

13 Hands-on and Review

Chapter: Chapter 4: Symbolic AI
Status: Drafted (V2.0, 3 Nov 2026). The advisor's forward and backward chaining runs were checked with a short program. Waiting for second-member review (see `tasks.md`, T16).
Owner / reviewer: see `tasks.md` (T10)
Prerequisites: Topics 07–12 (and the whole module for the review quiz).
Depth: Brief introduction (B) plus a hands-on project and a whole-module review.
Draft note: Prepared with Claude assistance (AI-use record AI-5). The rule runs were produced by code and compared with the hand traces.

Learning Objectives

After this topic, a beginner can:

1. Build a small **rule-based advisor** on paper: choose facts, write rules, and run forward and backward chaining.
2. Find and fix a **gap** in a knowledge base.
3. Discuss real **applications**, **advantages** and **limits** of Symbolic AI, and decide when it is a good choice.
4. Check their understanding of the whole module with a review quiz.

Explanation

Building a rule-based system in five steps

This brings together Topics 08 (IF-THEN logic), 09 (facts and symbols) and 10 (forward and backward chaining).

1. **Choose a narrow question.** Expert systems only work in a small domain. Example: *Which study method should I use tonight?*
2. **List the facts the user can supply.** Each must be checkable as true or false: "exam is within 3 days", "topic is new to me".
3. **Write IF-THEN rules.** Use **intermediate conclusions** (like "plan: final review") so rules can chain, just as "no power reaches the laptop" did in Topic 10.
4. **Run it.** Use forward chaining to see what the facts lead to, and backward chaining to check one particular piece of advice.
5. **Test and fix.** Try different combinations of facts. Where no advice comes out, the knowledge base has a **gap**; add a rule.

Where Symbolic AI is used today

Area	Example	Why symbolic methods fit
Rules set by law or policy	Tax calculators, benefit-eligibility checkers, insurance claim rules	The rules already exist in writing and every decision must be explainable
Configuration	Checking that a computer order or a car's options fit together (as XCON did)	Many exact compatibility rules

Area	Example	Why symbolic methods fit
Route finding and planning	GPS route search, delivery scheduling, timetabling, space-mission planning	Search over states and actions (Topic 11)
Games	Chess engines' search, game characters following rules	Clear rules and a clear goal
Checking correctness	Tools that prove a chip design or a program meets its specification	Logic gives guaranteed answers
Language tools	Grammar checkers, voice commands with a fixed vocabulary	Narrow, predictable language (Topic 12)
Hybrid AI	AlphaGo's tree search, chatbots that call a rule-based calculator or database	Combines learning with exact reasoning

Advantages and limits (summary of Chapter 4)

Advantages	Limits
Explainable: can show the rules and steps behind an answer (Topics 10, 11)	**Knowledge bottleneck:** experts must write every rule by hand (Topic 10)
Exact and consistent: same facts → same answer; logic is checkable (Topic 08)	**Brittle:** fails outside its rules or with a missing rule (Topics 07, 10)
Needs no training data	**Poor with messy input:** images, speech, handwriting, slang (Topic 12)
Easy to correct: edit one rule	**Hard to maintain** when rules number in the thousands and start to conflict
Good at search and planning with clear rules (Topic 11)	**Search can explode:** large state spaces have too many states to explore

When to choose Symbolic AI — a checklist. It is a good choice when most of these are true: the domain is narrow; experts can state the rules; decisions must be explained or audited; there is little training data; the input is already clean symbols (yes/no answers, numbers, database fields). If the input is raw images or free text, or nobody can state the rules, data-driven AI (or a hybrid) is usually better.

Worked Example: A six-rule study-method advisor

These rules are a practice example for reasoning, not tested study advice.

Knowledge base

Rule	IF	THEN
R1	exam is more than 3 days away AND topic is new	plan: spaced study
R2	exam is within 3 days AND topic is familiar	plan: final review
R3	exam is within 3 days AND topic is new	plan: focus on key points
R4	plan: spaced study AND learner likes diagrams	advice: make a concept map over several short sessions

Rule	IF	THEN
R5	plan: final review	advice: do practice questions from past tests
R6	plan: focus on key points AND studying with friends	advice: teach each other the main ideas

Facts (working memory at the start): F1 exam is more than 3 days away · F2 topic is new · F3 learner likes diagrams.

Forward chaining

Cycle	Rules whose conditions are all true	Rule fired	New fact
1	R1 (F1, F2). R2 and R3 fail: the exam is not within 3 days	R1	F4: plan: spaced study
2	R4 (F4, F3)	R4	F5: advice: make a concept map over several short sessions
3	None left	Stop	—

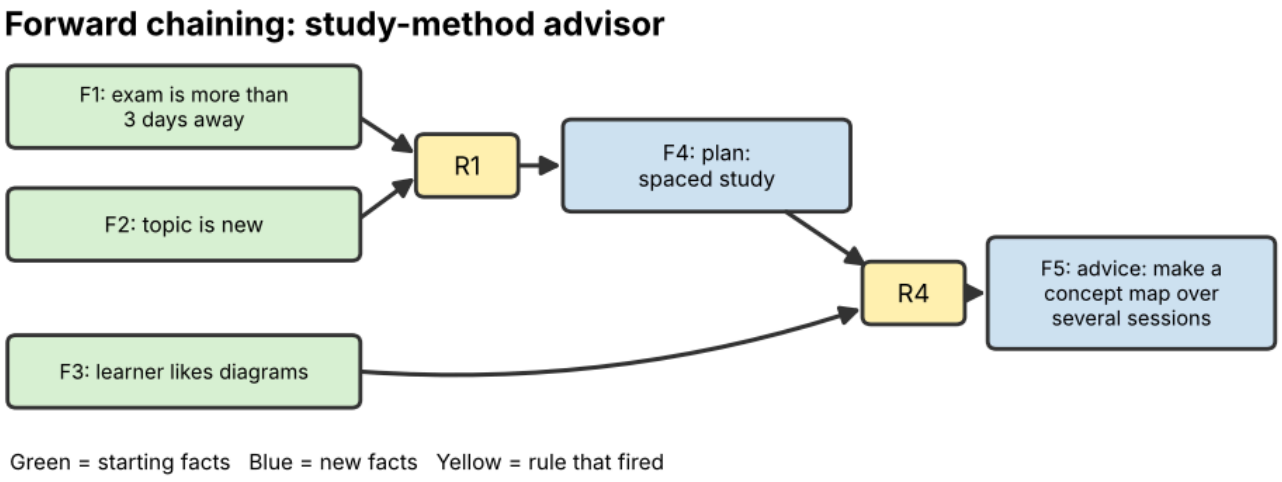


Figure 1. The rule chain: starting facts (green) → rule (yellow) → new facts (blue).

Backward chaining — goal: "advice: make a concept map over several short sessions".

1. R4 concludes it. Sub-goals: **plan: spaced study** and **learner likes diagrams**.
2. "Learner likes diagrams" is a fact (F3) ✔.
3. "Plan: spaced study" is not a fact; R1 concludes it. Sub-goals: **exam is more than 3 days away** (F1 ✔) and **topic is new** (F2 ✔).
4. All sub-goals proved → the goal is proved.

Activity

Part A: Hands-on with the advisor

1. **Run forward chaining.** New user: exam is **within 3 days**, topic is **new**, studying **alone**. Which rules fire? What advice is given?
2. **Fix the gap.** Write a rule R7 so that the user in question 1 gets advice. Run forward chaining again.
3. **Backward chaining.** Facts: exam is within 3 days, topic is familiar. Test the goal "advice: do practice questions from past tests". Write each sub-goal.
4. **Build your own.** Choose a new narrow question (for example, *Which transport should I take to the city centre?*). Write at least four facts and five rules, with at least one intermediate conclusion. Run forward chaining for one set of facts, and swap with a classmate to test each other's systems.

Part B: Module review quiz

Chapter 3: Software & Algorithms

1. (Topic 01) Is Weixin system or application software? Name the OS job that shares the CPU between Weixin and a music app.
2. (Topic 02) What does this print? `x = 2 · for i in range(3): ·  x = x * 2 · print(x)`
3. (Topic 03) A bug is found by real users only after release. Which testing level should have found that the app does not meet their needs?
4. (Topic 04) Using the Students table in Topic 04, what does `SELECT name FROM Students WHERE year = 2;` return?
5. (Topic 05) How many checks does binary search need, at most, for a sorted list of 1,000 items?
6. (Topic 06) Which structure suits customers waiting for a help-desk chat, and why?

Chapter 4: Symbolic AI

7. (Topic 07) Give one advantage of Symbolic AI over data-driven AI and one disadvantage.
8. (Topic 08) Rule: IF it is a weekend THEN the library opens at 10:00. Fact: the library opens at 10:00. Can you conclude it is a weekend? Name the pattern.
9. (Topic 09) Using the semantic network in Topic 09, does a canary breathe? Show the inheritance path.
10. (Topic 10) What is the difference in direction between forward and backward chaining?
11. (Topic 11) Which search method uses a queue, and what does that guarantee?
12. (Topic 12) Why can a grammar accept "The mat sat" even though it makes no sense?

Part C: Reflection

Write three to five sentences: *Describe one problem in your own life or major where Symbolic AI would work well and one where it would not. Use at least two items from the "when to choose Symbolic AI" checklist.*

Answer

Part A

1. R1 fails (the exam is not more than 3 days away). R2 fails (topic not familiar). **R3 fires** → plan: focus on key points. R6 needs "studying with friends", which is not a fact, so it fails. **No advice is given**: a gap in the knowledge base, like Activity 1 in Topic 10.

2. Example: **R7: IF plan: focus on key points AND studying alone THEN advice: make a one-page summary and test yourself on it.** Now R3 fires, then R7 fires and gives advice. Any rule that links "plan: focus on key points" (with or without "studying alone") to a sensible action is acceptable.
3. Goal → R5 → sub-goal **plan: final review** → R2 → sub-goals **exam is within 3 days** ✔ and **topic is familiar** ✔. All proved, so the advice is given.
4. Open answer. A good answer: (a) facts are checkable as true/false; (b) every rule has IF conditions and one THEN conclusion; (c) at least one rule's conclusion is used as another rule's condition; (d) the forward-chaining table shows each cycle, the rule fired and the new fact; (e) the classmate's test either produced advice or revealed a gap that was then fixed.

Part B

1. **Application** software. **Process management** (CPU scheduling).
2. 2 → 4 → 8 → 16. Prints **16**.
3. **Acceptance testing** (and, earlier, a better requirements stage).
4. **Bat** and **Chen**.
5. **10** checks.
6. A **queue**: first come, first served is fair (FIFO).
7. Advantage: it can **explain** its answers / needs no training data. Disadvantage: rules must be **written by hand** / it is **brittle** with messy input or missing rules.
8. **No.** The library might open at 10:00 on a holiday, too. This is **affirming the consequent**, an invalid pattern.
9. **Yes.** Canary → Bird → Animal; Animal stores "can: breathe", and the canary inherits it.
10. Forward chaining goes from **facts to conclusions** (data-driven); backward chaining starts from a **goal** and works back to the facts (goal-driven).
11. **Breadth-first search.** It explores level by level, so it finds a route with the **fewest steps**.
12. Grammar rules check **structure** (syntax), not **meaning** (semantics). "The mat sat" has the correct pattern Det N V.

Part C

Open answer. A strong reflection names a specific problem, links it to checklist items (narrow domain, rules can be stated, explanations needed, little data, clean symbolic input) and explains why the other problem fails the checklist (e.g. recognising plant diseases from photos: messy image input and rules hard to state, so data-driven AI fits better).

Common Misunderstandings

- "*A bigger knowledge base is always better.*" More rules mean more possible conflicts and harder maintenance. A small, tested rule set in a narrow domain often works better.
- "*Symbolic AI and machine learning are rivals; you must pick one.*" Many successful systems are hybrids that use each for what it does best.

Quick Self-Check

1. What do you call a situation where no rule produces advice for a valid set of facts? *(A gap in the knowledge base.)*
2. Name two kinds of task where Symbolic AI is a good choice. *(Any two: rule/law checking, configuration, route finding/planning, games with clear rules, correctness checking, fixed-vocabulary language tools.)*

How This Topic Will Be Reused

- **Study Guide / Test Bank:** final chapter; Part B questions seed the cumulative test bank (each tagged with its source topic).
- **PPT Slides:** five-step build method, study-advisor rules and Figure 1, applications table, advantages/limits table, checklist.
- **Video Tutorial:** not selected as a separate video; the advisor run may close tutorial 3 ("How a rule-based expert system reaches a conclusion") as a second example.
- **Live mini lecture:** the advisor is a backup in-class activity (the class acts as the inference engine).

References

1. Russell, S., & Norvig, P. (2021). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. (Applications and limits of knowledge-based systems; planning.)
2. Jackson, P. (1998). *Introduction to Expert Systems* (3rd ed.). Addison-Wesley. (Building and testing a knowledge base.)
3. Dunlosky, J., Rawson, K. A., Marsh, E. J., Nathan, M. J., & Willingham, D. T. (2013). Improving students' learning with effective learning techniques. *Psychological Science in the Public Interest, 14*(1), 4–58. (Background for the study-method example: spaced and practice-test techniques.)
4. Foundations of CS & AI 2026, Chapter 3 and Chapter 4 course slides.

The study-method advisor, quiz questions and reflection task are the team's own.